Chording and Tilting for Rapid, Unambiguous Text Entry to Mobile Phones

by

Daniel J. Wigdor

A thesis submitted in conformity with the requirements
for the degree of Masters of Science
Graduate Department of Computer Science
University of Toronto

Copyright © 2004 by Daniel J. Wigdor

ABSTRACT

Chording and Tilting for Rapid, Unambiguous Text Entry to Mobile Phones

Daniel J. Wigdor
Masters of Science
Graduate Department of Computer Science
University of Toronto
©2004

The numeric keypads on mobile phones generally consist of 12 keys, and thus require multiplexing to use them to enter the 36-characters of the English alphabet and decimal numbers. There exist several techniques for entering text using this keypad, but none has emerged as a clearly superior technique. Chording text entry, where users are required to press multiple keys simultaneously, has been shown to be a superior approach to text entry for mobile systems, although it has failed to be adopted widely. We present two new text-entry systems for mobile phones that use chording: *ChordTap*, which uses chord keys, and *TiltText* which replaces chording keys with tilting gestures. we present the results of controlled experiments, which show both *ChordTap* and *TiltText* to be respectively 45% and 22% faster than the traditional *MultiTap*, despite a higher error rate for *TiltText* (9% vs 3% for *ChordTap* and *MultiTap*).

ACKNOWLEDGEMENTS

My path into research in computer science was a more or less direct one, beginning with my experiences learning to program computers with the great help of Richard Watson and Jason Wilson at Uxbridge Secondary School. Without them, I would never have been able to pursue computer science as an undergraduate discipline.

It was dr. monica schraefel, who took a chance on an inexperienced undergraduate research assistant, and opened my eyes to research in Human Computer Interaction. Without her assistance and quick confidence in me, I would never have been able to start work in HCI. More recently, a challenge from Professor Ravin Balakrishnan, now my supervisor, to "solve" text entry into mobile phones launched my research career and MSc work. His expectations have continually set the bar higher, and his enthusiasm and subtle assistance have allowed me to vault it. For your guidance and support, I am forever grateful. I also thank Ron Baecker, professor of HCI, at U of T, for helping me along the way, and with this thesis. The DGP lab at the University of Toronto is home to a unique mix of great scientists and artists. The many great minds there have challenged and aided every step of my research, and made it a wonderful experience to come to work each day. Thank you.

Of course, I could never have pursued academics without the love and support of William who challenges, Jason and Colin who humble, Adam and Noel who reflect, and especially my parents, Robin and Irene who guide and push. Most of all, I thank Maya for opening my eyes to new beauty and creativity I had never imagined. I love you all.

Much of the material included in this thesis has previously been published in journals of the ACM, and has been included herein with their permission[1] and permission of my co-author:

Wigdor, D., Balakrishnan, R. (2004 - in press) A Comparison of Consecutive and Concurrent Input Text Entry Techniques for Mobile Phones. *CHI Conference on Human Factors in Computing Systems*. (8).

Wigdor, D., Balakrishnan, R. (2003) TiltText: Using tilt for text input to mobile phones. *Proceedings of the 16th Annual ACM UIST Symposium on User Interface Software and Technology*. (p 81-90).

TABLE OF CONTENTS

ABSTRACT ..... **II**

ACKNOWLEDGEMENTS ..... **III**

TABLE OF CONTENTS ..... **IV**

TABLE OF FIGURES ..... **VII**

TABLE OF TABLES ..... **IX**

FOOTNOTES ..... **IX**

1 INTRODUCTION **1:1**

2 BACKGROUND **2:5**

2.1 CONSECUTIVE KEYPRESS INPUT 2:5

2.1.1 Words per Minute (WPM) Metric *2:5*

2.1.2 Keystrokes per Character (KSPC) Metric *2:5*

2.1.3 Benchmark: QWERTY Keyboard *2:6*

2.1.4 Small QWERTY Keypads *2:6*

2.1.5 Non-Traditional Mobile-Phone Keypads *2:7*

2.1.6 On-Screen Character Selection *2:8*

2.1.7 Traditional Phone Keypad *2:9*

2.2 CONCURRENT KEYPRESS TECHNIQUES 2:13

2.2.1 Chording Keyboards *2:13*

2.2.2 Performance of Chording Keyboards *2:13*

2.2.3 Mobile Chording Keyboards *2:14*

2.3 USING TILT SENSORS IN MOBILE DEVICES 2:16

3 CHORDING INPUT FOR MOBILE PHONES **3:18**

3.1 PLACING THE CHORDS 3:20

3.2 MAPPING CHORDS STATES TO WITHIN-GROUP SELECTION 3:21

3.3 EVENT HANDLING 3:22

3.3.1 Treating Only Chord Presses as Events *3:22*

3.3.2 Treating Only Keypad Presses as Events *3:23*

3.3.3 Both Chord & Keypad Presses as Events *3:23*

3.4 PROTOTYPE 3:24

3.4.1 Hardware *3:24*

3.4.2 Software *3:25*

4 USER STUDY COMPARING EARLY LEARNING STAGE OF CHORDTAP AND MULTITAP 4:26

4.1 GOALS 4:26
4.2 APPARATUS 4:26
4.3 PARTICIPANTS 4:27
4.4 PROCEDURE 4:27
4.5 DESIGN 4:30
4.6 RESULTS 4:31
4.6.1 Physical Comfort 4:31
4.6.2 Overall Entry Speed 4:31
4.6.3 Learning 4:32
4.6.4 Error Rates 4:33
4.7 DISCUSSION 4:35

5 A NEW TECHNIQUE: TILTTEXT 5:36

5.1 DESIGN ISSUES 5:38
5.2 TECHNIQUES FOR CALCULATING TILT 5:39
5.2.1 Key Tilt 5:39
5.2.2 Absolute Tilt 5:39
5.2.3 Relative Tilt 5:40
5.3 PROTOTYPE 5:40
5.3.1 Hardware 5:40
5.3.2 Handedness 5:42
5.3.3 Software 5:42

6 USER STUDY COMPARING EARLY LEARNING STAGE OF TILTTEXT, MULTITAP, AND CHORDTAP 6:43

6.1 GOALS 6:43
6.2 PARTICIPANTS 6:44
6.3 PROCEDURE 6:44
6.4 DESIGN 6:46
6.5 RESULTS 6:47
6.5.1 Data Summary 6:47
6.5.2 Physical Comfort 6:47
6.5.3 Text Entry Speed vs MultiTap 6:47
6.5.4 Text Entry Speed: TiltText vs ChordTap 6:50
6.5.5 Error Rates: TiltText vs MultiTap 6:51
6.5.6 Error Rates: TiltText vs ChordTap 6:55

6.6 DISCUSSION 6:56

7 CONCLUSIONS AND FUTURE WORK 7:59

7.1 CONCLUSIONS 7:59

7.2 CONTRIBUTIONS 7:59

7.3 FUTURE DIRECTIONS 7:60

7.3.1 ChordTap 7:61

7.3.2 TiltText 7:61

8 BIBLIOGRAPHY 8:62

TABLE OF FIGURES

Figure 1–1: Standard mobile phone keypad 1:1

Figure 1–2:Xerox PARC Version of Engelbart & English Chording Keyboard 1:3

Figure 2–1: Miniature QWERTY keypads from Handspring Treo 600 (left) and Nokia 3300 (www.handspring.com, www.nokia.com). 2:7

Figure 2–2: Nokia 3600 with its circular keypad 2:8

Figure 2–3: Nokia N-Gage with keypad on the right side and landscape orientation. 2:8

Figure 2–4. Standard 12-key mobile phone keypad 2:9

Figure 2–5: The Englebart Chording Keyboard, and QWERTY Keyboard. 2:14

Figure 2–6: Twiddler2 (left) and Septambic Keyer: one-handed chording keyboards. 2:15

Figure 2–7: Half-QWERTY keyboard, built by Matias Corporation (www.matias.com). 2:15

Figure 3–1: ChordTap prototype. The right image shows the chord keys mounted on the back of the phone. 3:19

Figure 3–2: ChordTap, as used by a left-handed (left) and right-handed user. 3:20

Figure 3–3: circuit diagram of ChordTap prototype. 3:25

Figure 3–4: ChordTap text field within Motorola GUI form. 3:25

Figure 4–1: Emulation of phone as shown to user. Example instructions (left), and timed text entry portion (right). 4:27

Figure 4–2: MultiTap as used with left, right, and both hands. 4:29

Figure 4–3: Entry speed (WPM) by technique and block for entire experiment. Best-fit power law of learning curve shows projected progress beyond the 16 blocks of measured data. 4:32

Figure 4–4: Error rate per 100 attempted character entries by block for all three techniques. 4:33

Figure 4–5: Key and Chord error rates per 100 attempted entries for each character in the experiment (space shown as “>”). 4:34

Figure 4–6: Chord error rate by required chord. Since all multiple-chords (011,101,110,111) produced the same letter in our prototype, they are combined in this graph. 4:35

Figure 5–1: TiltText. The center picture shows the untilted phone where pressing a key enters its numeric value. Left picture: left tilt enters first character on key. Top picture: forward tilt enters second character. Right picture: right tilt enters third character. Bottom picture: tilting back (towards the user) enters fourth character if one exists for that key. 5:37

Figure 5–2: Uppercase text entry with TiltText. Tilting beyond a threshold makes the character uppercase. 5:38

Figure 5–3: TiltText prototype circuit diagram. 5:41

Figure 5–4: TiltText prototype held with left and right hands. 5:42

Figure 5–5: GUI built in Motorola framework, including a TiltText enabled field. 5:42

Figure 6–1: Entry speed (WPM) by technique and block for entire experiment. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks. 6:49

Figure 6–2: Entry speed (WPM) by technique and block, for the first half of the experiment, before participants switched techniques. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks. 6:49

Figure 6–3: Entry speed (WPM) by technique and block, for the second half of the experiment, after participants switched techniques. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks. 6:50

Figure 6–4: WPM rate of ChordTap vs TiltText over the 16 blocks of the experiment. 6:50

Figure 6–5: Difference in speeds between ChordTap and TiltText over the 16 blocks of the experiment. 6:51

Figure 6–6: Error rates (%) by technique and block for entire experiment 6:53

Figure 6–7: Error rates (%) by technique and block for the first half of the experiment, before participants switched techniques 6:53

Figure 6–8: Error rates (%) by technique and block for the second half of the experiment, after participants switched techniques 6:54

Figure 6–9: Tilt error rates (%) for each letter for the entire experiment. 6:54

Figure 6–10: Error rates (%) by direction of tilt for correct letter for the entire experiment. 6:55

Figure 6–11: Error rate of TiltText vs ChordTap 6:56

TABLE OF TABLES

Table 3-1: Mapping of chord state to within-group characters. Example selection shown based on pressing the 7 key. 3:21

Table 3-2: Sequence of actions required to enter the string "only" in a ChordTap implementation that treats only chord presses as events. Some consecutive actions are combined because they either generate no text, or the same text is generated with either ordering. 3:22

Table 3-3: Sequence of user actions required to enter the string "only" in a ChordTap implementation that treats only keypad presses as events. Some consecutive actions are combined because they either generate no text, or the same text is generated with either ordering. 3:23

Table 3-4: Sequence of actions required to enter the string "only" in a ChordTap implementation that treats both chord and keypad presses as events. Note that ordering of events required to enter text is not unique. 3:24

Table 4-1:Average speed (WPM) for each technique over the two days of the study 4:32

FOOTNOTES

1: The ACM copyright notice, below, applies only to those sections previously published in journals of the ACM (see Bibliography, Wigdor: 2003, Wigdor: 2004):

CM COPYRIGHT NOTICE. Copyright © 2001, 2002, 2003 by the Association for Computing Machinery, Inc. Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Publications Dept., ACM, Inc., fax +1 (212) 869-0481, or permissions@acm.org. © 2001, 2002, 2003 ACM, Inc. Included here by permission

1 Introduction

Most mobile phones are equipped with a simple 12-button keypad, shown in Figure 1-1, which is an inherently poor tool for generating phrases for a 26-letter alphabet. It is therefore surprising that nearly 500 billion text messages were estimated to have been sent worldwide from mobile phones in 2003 (Wigdor and Balakrishnan 2003). Given the persistent popularity of the traditional keypad, there is a need to invent techniques for efficiently entering text using this restricted set of keys. The traditional, and most common technique, *MultiTap*, works by requiring multiple, consecutive presses of the keys to generate each character of text. It has been shown that, on average, roughly 2 keystrokes are required for each character of text generated using *MultiTap* (MacKenzie 2002). Thus, for a simple 7 word message with 5 character words, the user must make approximately 70 keypresses.

Figure 1–1: Standard mobile phone keypad

Several approaches have been presented that overcome this problem. Smaller versions of the standard QWERTY keypad have been built-into mobile phones, and have been shown to be effective for speedy text entry (MacKenzie and Soukoreff 2002). They have, however, failed to gain acceptance. Approaches that speed text entry using the traditional keypad have thus been the focus of research in this area (MacKenzie, Kober et al. 2001),(Soukoreff and MacKenzie 2002),(Silfverberg, I. Scott MacKenzie et al. 2000). Mackenzie et al. (MacKenzie and Soukoreff 2002) describe this problem as involving two main tasks necessary for entering a character: *between-group* selection of the appropriate group of characters, and *within-group* selection of the appropriate character within the previously chosen group.

Most text input techniques to date can generally be divided into two categories: those that require multiple presses of a single key to make the between-group followed by within-

group selections, and those that require a single press of multiple keys to make these selections. Because both categories require consecutive key presses, the research focus has been on reducing the average number of key strokes per character "KSPC" required to enter text. Advances in the area generally make language specific assumptions to "guess" the desired within-group character, thus reducing or eliminating the key presses required for the within-group selection. Many of these techniques show strong improvement over *MultiTap* (Silfverberg, I. Scott MacKenzie et al. 2000), (MacKenzie and Soukoreff 2002), but all share its most critical flaw: multiple keystrokes must be made consecutively to generate each character of text. Though all reduce the number of keystrokes required, this reliance upon multiple, consecutive presses create a constant ceiling on potential performance. What's worse, many of these techniques only reduce the number of keystrokes when text from a particular language is entered. When attempting to enter words not present in the phone's built-in dictionary, the number of keystrokes required is actually much greater than with *MultiTap* (MacKenzie and Soukoreff 2002).

Outside the domain of mobile phones, there exist several text-entry systems, generally classified as "chording" techniques, which also generate text from a large alphabet using a smaller number of keys (Conrad and Longman 1965). Chording requires the user to press multiple keys simultaneously to generate each character of text. Although multiple keystrokes are required for each character, performance is enhanced because these presses are made concurrent to one another, saving time. Because different combinations of key presses can be used to generate a character of text, only a small number of keys is required to allow unique identification of the entire roman alphabet. For example, a 5-key chording keyboard has 2^5-1 = 31 different keypress combinations, and so can be used to enter the standard Roman alphabet.

Chording keyboards were reported as early as 1942 (Conrad and Longman 1965), and a chording keyboard designed to be used with one hand was included in Englebart and English's presentation of the Augmented Knowledge Workshop (Engelbart and English 1968) in the 1960's. Their chording keyboard, as shown in Figure 1-2, which resembles

a piano keyboard, was meant to be used simultaneously with mouse manipulation (Engelbart 1986).

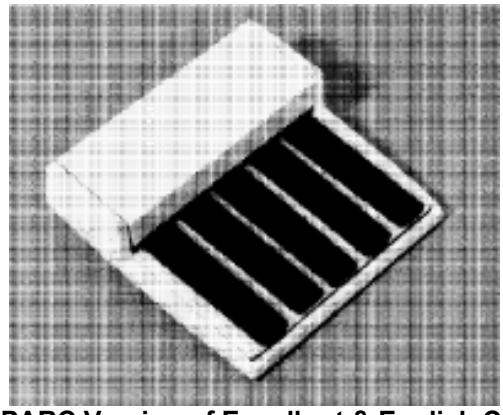

Figure 1–2:Xerox PARC Version of Engelbart & English Chording Keyboard

Subsequently, two-handed chorded keyboards have been used by the US postal service for mail sorting (Rosenberg 1994), and are still used today by stenographers.

Chording keyboards exist for mobile applications, and adapting these for mobile phones is trivial. As we have seen with the failure of QWERTY-equipped phones to gain wide adoption, consumers seem reluctant to abandon the standard mobile phone keypad. How, then, can we apply the tried and tested chording approach to the mobile phone? This question drove us to develop two new mobile-phone text entry techniques that use chording. In this thesis, we first present *ChordTap*, which makes use of additional keys mounted on the back of the phone. Users are still required to make the same two selections identified by MacKenzie (MacKenzie and Soukoreff 2002), but they are made concurrently: the between-group selection is made with the mobile phone keypad, while simultaneously the within-group selection is made using the chord keys.

In order to examine its effectiveness, we conducted a controlled experiment of *ChordTap*. We pitted it against the standard *MultiTap* technique, tested by two groups of users, one using just one hand and the other using both hands to enter text. We examined both is speed of entry and error rates, and found that it was significantly better than both *MultiTap* approaches.

In order to simplify its construction and use, our *ChordTap* prototype was designed to be used with both hands – one used to press keypad keys, the other to press chord keys. We felt that, despite its laboratory success, its reliance on two-handed use limited its applicability to a real-world solution. In alleviation of this problem, we then present *TiltText*, a technique which allows the user to chord with one hand, using gestures in place of chord keys. Like *ChordTap*, between-group character selections are made by pressing standard keypad keys. The user simultaneously makes the within-group character selection by tilting the phone in one of four directions, in order to select from among the three or four possible characters within the group.

We conducted another controlled experiment, this time pitting *TiltText* against *MultiTap*. The experimental design was identical to that used for *ChordTap*, allowing us to make direct comparisons between *TiltText* and *ChordTap*. We found that *TiltText* was faster than *MultiTap* despite a much higher error rate, but not as fast as *ChordTap*. We offer an explanation as to the hampered performance, and present possible options to overcome it.

We conclude by discussing the implications and future directions of our research.

2 Background

As background, we first present research into mobile-phone text entry, then discuss previous work in chording for text entry, and tilting in mobile devices.

2.1 Consecutive Keypress Input

The keyboard is a tried and trusted input device for desktop computers. Because buttons are inexpensive, there is motivation for the manufacturers to keep this approach for mobile devices. Due to the inherent size limitations for input to mobile phones, it is impossible for them to have a full-sized built-in QWERTY keyboard. There are a number of solutions to this problem, one of which, the telephone keypad, is the primary focus of this thesis.

We will review key-based small device input solutions, with a focus on the multiplexed keyboard approaches used in most mobile phones.

2.1.1 Words per Minute (WPM) Metric

The traditional measure of an individual's performance in text entry is the words per minute (WPM) metric. WPM is measured by examining the number of characters a user entered, and the time they took to enter those characters. The number of characters per minute is calculated first, then divided by 5, which is taken to be the average length of a word (Silfverberg, I. Scott MacKenzie et al. 2000). The effectiveness of a technique for text entry is often measured by the average speed of entry by some group of users, measured in words per minute.

2.1.2 Keystrokes per Character (KSPC) Metric

Because multiple keystrokes are necessary to enter an unambiguous character of text on a multiplexed keyboard, researchers in this area rely heavily on keystrokes per character (KSPC) metric (MacKenzie 2002). New techniques are generally evaluated by this metric, and thus the focus of research to date has been to reduce the number of keystrokes required, on average, to enter a character of text.

There are generally two ways to calculate KSPC. The first is to take the average number of button presses required to enter each of the characters in the alphabet using a particular technique. The second approach is to measure the number of keystrokes required to enter each word in some corpus, then take the average number of keystrokes required to enter each character. The second approach has two advantages: first, it allows the KSPC metric to be applied to techniques that render characters based on the context of the surrounding word(s). It also allows for a more "realistic" evaluation of a technique, presuming that the corpus used is representative of that which users draw from in general use.

2.1.3 Benchmark: QWERTY Keyboard

The classic computer keyboard has one key for each character in the English alphabet, with the keys laid out in three rows of 7-10 keys. Such keyboards are usually reported as having a KSPC of 1 (MacKenzie and Soukoreff 2002; Wigdor and Balakrishnan 2003), though more than one keystroke is generally required to enter upper-case letters and symbols. Several alternative layouts have been seen, but the QWERTY layout is the standard for keyboards in use today. Though there are advantages to these other layouts (West 1998), such as the DVORAK configuration, user studies have shown that these advantages are minimal (Norman and Fisher 1982), although Zhai (Zhai and Smith 2001) suggests that layout matters more for virtual keyboards. Typing speed can vary widely among users, but several studies (Matias, MacKenzie et al. 1993; Matias, MacKenzie et al. 1996; West 1998; Ward, Blackwell et al. 2000) show that skilled typists generally achieve speeds in excess of 60 words per minute, and are sometimes seen to exceed 80-100 words per minute. Because of this high speed of text entry, and their ubiquity in computing environments, QWERTY is generally seen at the benchmark for text-entry research.

2.1.4 Small QWERTY Keypads

Because of its popularity in other domains, the most obvious input device for mobile applications is the QWERTY keyboard. Due to the inherent size limitations of the

mobile-phone domain, it is impossible to include a full-sized QWERTY keypad. Several mobile phones have been designed that use a miniature version of the QWERTY keypad, with the same layout but less densely packed, smaller keys, as shown in Figure 2-1.

Figure 2–1: Miniature QWERTY keypads from Handspring Treo 600 (left) and Nokia 3300 (www.handspring.com, www.nokia.com).

Because of their smaller size, most users cannot use all ten fingers simultaneously to enter text, as is typically done on full-sized QWERTY keyboards. Interestingly, MacKenzie and Soukoreff (MacKenzie and Soukoreff 2002) found that the miniature QWERTY keypad is quite well suited for two-thumb text entry, allowing users to enter text at quite high speeds when typing with two thumbs. They developed a model which predicted that peak speed for these keyboards is 60.74 WPM, or roughly 4 characters per second.

Given the high speed of text entry possible with these devices, it is somewhat surprising that they are not more popular. We are aware of no academic research that explains this phenomenon, but it is reasonable to assume that market factors have driven manufacturers to make use of the traditional mobile phone keypad in creating new designs.

2.1.5 Non-Traditional Mobile-Phone Keypads

A number of devices have been built with non-traditional keypads. Though no formal evaluation of these devices has been conducted, they are included here for completeness.

The Nokia 3600, shown below, uses the same character groupings as the traditional keypad shown in Figure 1-1, arranged in a circular pattern. We are aware of no formal evaluation of performance using this arrangement.

Figure 2–2: Nokia 3600 with its circular keypad

The Nokia N-Gage (Figure 2-3) is a device designed to be used as both a phone and video-game machine. It uses a keypad similar to the traditional mobile phone keypad. The interaction is changed somewhat, however, by the location of the pad. This orientation would force users to enter text with only one hand, and not hold the phone in the same way others are held. We are aware of no formal evaluations of text-entry performance of the N-Gage.

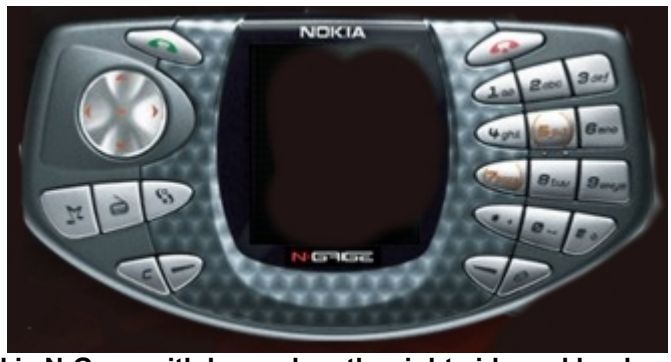

Figure 2–3: Nokia N-Gage with keypad on the right side and landscape orientation.

2.1.6 On-Screen Character Selection

On-screen character selection systems typically employ one, two, or four navigation keys and a selection key. Used in a variety of applications, these systems work by requiring the user to scroll through a list of available characters, presented on a screen, using the navigation key(s) and selecting the desired character with the selection key once reached.

MacKenzie (MacKenzie 2002) presented six different techniques for entering text using just three keys, including one making use of "language enhancement". The enhancement he proposed was to re arrange the list of characters each time the user made a selection, so that probable next characters were placed closer to the current cursor position. This approach dramatically reduced the KSPC rate for the three-key techniques (to 4.23 from over 10 for most others), a user study found that this was no faster than non linguistically optimized techniques in practice. This was not surprising, given that a dynamic character list makes learning next-to impossible.

2.1.7 Traditional Phone Keypad

Mobile phones typically use the same keypad that was presented with the very first touchtone telephones. A 4 x 3 matrix of keys, with primary labels of 0-9, *, and # (see Figure 2-4).

Figure 2–4. Standard 12-key mobile phone keypad

Entering text from a 26 character alphabet using this keypad forces a mapping of more than one character per button of the keypad. A typical mapping has keys `2-9` representing either three or four characters, with space and punctuation mapped to the other buttons. All text input techniques that use this standard keypad have to somehow resolve the ambiguity that arises from this multiplexed mapping. There are three main techniques for overcoming this ambiguity: *MultiTap*, two-key, and linguistic disambiguation. In reviewing these technologies, we will focus on the Key Strokes Per Character (KSPC) metric, which is most often used to demonstrate the effectiveness of new techniques.

2.1.7.1 MultiTap

MultiTap works by requiring the user to make multiple presses of each key to indicate which letter on that key is desired. For example, the letters `pqrs` traditionally appear on the `7` key. Pressing that key once yields `p`, twice `q`, etc. A problem arises when the user

attempts to enter two consecutive letters on the same button. For example, tapping the 2 key three times could result in either c or ab. To overcome this, *MultiTap* employs a time-out on the button presses, typically 1-2 seconds, so that not pressing a button for the length of the timeout indicates that you are done entering that letter. Entering ab under this scheme has the user press the 2 key once for a, wait for the timeout, then press 2 twice more to enter b. To overcome the time overhead this incurs, many implementations add a "timeout kill" button that allows the user to skip the timeout. If we assume that 0 is the timeout kill button, this makes the sequence of button presses to enter ab: 2,0,2,2. *MultiTap* eliminates any ambiguity, but can be quite slow, with a keystrokes per character (KSPC) rate of approximately 2.03 (MacKenzie, Kober et al. 2001).

2.1.7.2 Two-key Disambiguation

The two-key technique requires the user to press two keys in quick succession to enter a character. The first keypress selects the appropriate group of characters, while the second identifies the position of the desired character within that group. For example, to enter the character e, the user presses the 3 key to select the group def, followed by the 2 key since e is in the second position within the group. This technique, while quite simple, has failed to gain popularity for Roman alphabets. It has an obvious KSPC rate of 2.

2.1.7.3 Linguistic Disambiguation

Linguistic disambiguation attempts to reduce keypresses by using knowledge of the input language of the user to select the most probable character from within the group specified by the keypress. There are several such techniques; the most commonly found in mobile phones is T9, developed by Tegic Communications Inc. A dictionary is stored in the phone which constitutes the set of words from which the user is able to select.

In T9, the "space" key is unambiguous. For each keystroke (i) in a word of length n, the system retrieves from its dictionary all word beginnings that are rendered using those i keystrokes. After each lookup, the display shifts to show that prefix which is most probable (based on probabilities pre-set by the manufacturer). On the nth keystroke, if the

display does not match the intended word, the user presses a “next” key to cycle through all possible words rendered with those n keystrokes. To save keystrokes, the words are presented to the user in order, from most probable to least probable.

One consequence of the T9 system is display instability. Because n lookups are done when entering a word of length n, there is the potential that the display will change n times while rendering the word. This instability can be confusing, and makes errors difficult to track (since it is normal that the wrong letter is rendered while entering each character).

T9’s greatest flaw is that it is impossible to render words not present in its dictionary. In most cases, the user is able to switch to another mode (such as *MultiTap*) to enter these words, and then switch back to T9 mode. In some T9 implementations, these new words are added to the built-in dictionary automatically, improving future performance.

MacKenzie (MacKenzie and Soukoreff 2002) performed an analysis of how often the “next” key is pressed by a user entering only those words that appear in the dictionary (the dictionary used was a standard corpus, rather than T9’s, for legal reasons). They found that the KSPC measure was only 1.0072, indicating that “next” is entered only infrequently. This was calculated under an assumption of uniformity of frequency of words from the corpus, and that only those words in the corpus would be entered. Grinter & Eldridge(Grinter and Eldridge 2001) point out that this assumption is probably not realistic.

To improve over T9, MacKenzie et al developed *Letterwise* (MacKenzie, Kober et al. 2001). Like T9, text is rendered by looking-up word prefixes from keystrokes since the last “space”. *Letterwise* differs in that the prefixes are generated based on letter frequencies (“e” is more likely to follow “th” than are “d” or “f”), rather than dictionary look-up, and in that each character is rendered individually, rather than reconsidering the prefix as a whole on each keystroke. This approach has three main advantages: “English-like” words can be rendered, thus the language is not strictly limited to a set dictionary;

less memory is required in the device, because no dictionary needs to be stored; and because each character is looked-up only once, there is no display instability. The main disadvantage is that, because each character is generated based only on preceding characters (rather than looking up the word as a whole on each keystroke), the "next" key must be pressed to correct each letter. MacKenzie et al's analysis concluded that only 50.1% of words could be rendered without pressing "next" for at least one of the characters. Among the 50.1% of words, only some letters required disambiguation, giving *LetterWise* a KSPC rate of 1.150.

Another language-based technique is *WordWise* (MacKenzie and Soukoreff 2002), which, like T9, uses dictionary lookup to render text. What distinguishes *WordWise*, and what makes it of particular interest to the present research, is that it uses chording to enter some characters unambiguously.

This is accomplished by identifying a particular character in each grouping associated with a key. When the user presses a chord key in combination with a keypad key, the unambiguous character is entered. If the chord key is not pressed, one of the remaining characters from the group is rendered, based on the surrounding context of unambiguous characters. The choice characters, {'c','e','h','l','n','s','t','v'} were chosen to minimize query and lookup error rates (MacKenzie and Soukoreff 2002). The query rate is how often the system must use the dictionary to render a word, and the lookup error rate is the rate at which the desired word is not the first one rendered (thus requiring the press of a "next" key). This is the only technique in the literature to date that used chording for text entry to mobile phones.

All language-based text entry techniques report excellent KSPC rates, many approaching the seemingly ideal 1.0. All, though, achieve this high speed at the high cost of requiring the user to enter text that conforms to a language. T9 and *WordWise* require that every word come from a standard dictionary, while *LetterWise* can render any "English like" words. As MacKenzie et al. note (MacKenzie, Kober et al. 2001), users often use abbreviations, and not complete English when text messaging. Further, users of text

messaging often communicate in acronyms or combinations of letters and numbers (e.g., `b4` for `before`). Another problem with these linguistic techniques is that users have to visually monitor the screen in order to resolve potential ambiguities, whereas the *MultiTap* and two-key techniques can be operated "eyes-free" by skilled users.

As a result of these limitations of current keypad text input techniques, the quest for a widely applicable, low KSPC, text input technique continues.

2.2 Concurrent Keypress Techniques

2.2.1 Chording Keyboards

A chording keyboard is one where characters are entered using combinations of key presses. For example, a chording keypad with 2 keys could be used to enter 3 unique characters: "a" when the first key is pressed, "b" when the second key is pressed, and "c" when both buttons are pressed. Reported as early as 1942 (Conrad and Longman 1965), chording keyboards have been explored in various dimensions and configurations, and we now briefly review this literature.

2.2.2 Performance of Chording Keyboards

If characters are mapped to all possible key press combinations, a simple one-handed five key chord keyboard can enter 31 ($2^5 - 1$) distinct characters – for many text applications, this is sufficient. Adding the second hand increases this to 1023 ($2^{10} - 1$) unique characters. By limiting the number of keys to the number of fingers used to enter text, time is saved by eliminating the need to move the fingers to different keys. The trade-off for this speed enhancement, as discussed by Norman et al, (Norman and Fisher 1982) is that a chording keyboard does not have the affordances for use that a keyboard with a 1:1 mapping of letters to keys does. Consider the keyboards shown in Figure 2-5. The Englebart chording keyboard is capable of entering the same letters of the alphabet as the QWERTY, also shown. The QWERTY version, however, has a clear mapping of key to button, not possible on the chording keyboard.

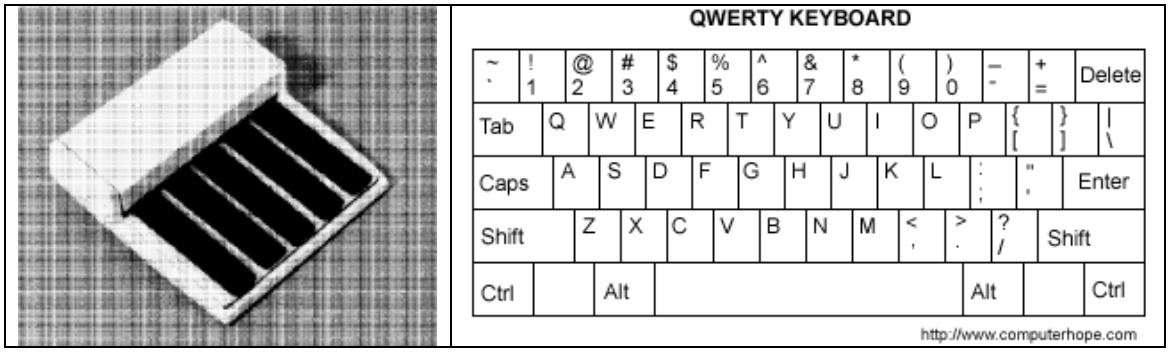

Figure 2–5: The Englebart Chording Keyboard, and QWERTY Keyboard.

Conrad and Longman (Conrad and Longman 1965) found that, with training, chording keyboards are faster and easier to learn than traditional keyboards. Gopher and Koenig (Gopher and Koenig 1983) examined how best to determine the optimal mapping of chordings to characters of text. Gopher and Raij (Gopher and Raij 1988) examined whether the two-handed chording keyboard had any advantage over a one-handed implementation. They found that while both significantly outpaced a QWERTY keyboard, there was no significant difference in performance between their one and two-handed chording keyboards in the early stages of learning. As average user speed started to approach 32 WPM, the two-handed keyboard started to outperform its one-handed counterpart, and this spread in performance continued to grow as users gained more experience.

2.2.3 Mobile Chording Keyboards

The *Twiddler* (www.handykey.com) (Figure 2-6) and the *Septambic Keyer* (wearcam.org/septambic/) are examples of modern-day one-handed chording keyboards. Designed to be held in the hand while text is being entered, both are commonly used as part of a wearable computer (Barfield and Caudell 2001). The *Twiddler* is equipped with 6 keys to be used with the thumb, and 12 for the fingers, while the traditional *Septambic Keyer* has just 3 thumb and 4 finger switches.

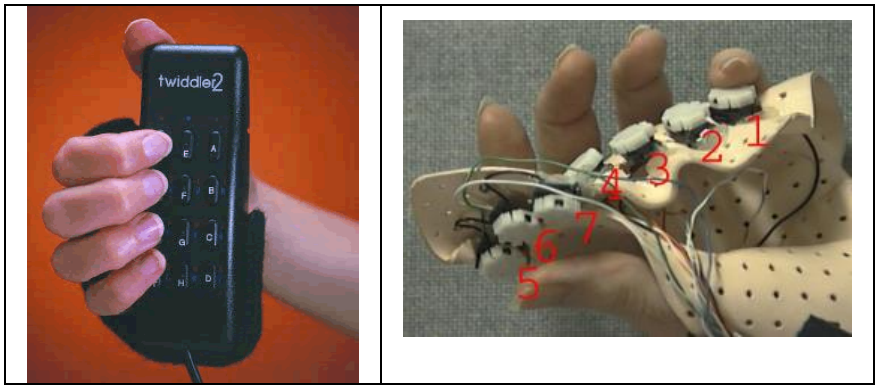

Figure 2–6: Twiddler2 (left) and Septambic Keyer: one-handed chording keyboards.

The Septambic Keyer allows for 47 different combinations of key presses, while the *Twiddler* allows over 80,000, though not all keys are used for text entry.

We are not aware of any published evaluations of the performance of these keyboards, but their popularity within the wearable-computer community suggests that there is potential for a chording-based technique for mobile phones.

Another chording keyboard, the 1/2 QWERTY was presented by Matias et al (Matias, MacKenzie et al. 1996). The system used half the usual number of keys for a QWERTY keypad, and required the user to press the space-bar prior to entering those keys that are normally located on a particular side of the keyboard. The results of their controlled experiment showed quick adaptation by expert users. Matias Corporation now manufactures a half-qwerty keypad, shown in Figure 2-7. Though this is only a very basic implementation of chording, it too points to the potential of a chording-based solution for mobile phone text entry.

Figure 2–7: Half-QWERTY keyboard, built by Matias Corporation (www.matias.com).

2.3 Using Tilt Sensors in Mobile Devices

Several researchers have recently proposed interesting interaction techniques that are enabled by incorporating a low-cost tilt sensor within mobile devices (Rekimoto 1996; Harrison, Fishkin et al. 1998; Schmidt, Aidoo et al. 1999; Bartlett 2000; Fishkin, Gujar et al. 2000; Hinckley, Pierce et al. 2000; Hinckley and Horvitz 2001; Partridge, Chatterjee et al. 2002; Sazawal, Want et al. 2002). While some of this prior art (e.g., (Harrison, Fishkin et al. 1998; Schmidt, Aidoo et al. 1999; Bartlett 2000; Fishkin, Gujar et al. 2000; Hinckley, Pierce et al. 2000; Hinckley and Horvitz 2001) (Rekimoto 1996)) do not concern text entry techniques per se, they do add to the set of possible interactions that could take advantage of tilt sensors embedded in mobile devices, thus providing further justification for the incremental cost of the sensor.

In particular, Hinckley et al's (Hinckley, Pierce et al. 2000) review of how to including tilting in an event-driven system provides clear context for Fishkin et al's (Fishkin, Gujar et al. 2000) review of document navigation, list traversal, and document annotation techniques that make use of tilt interaction, and Hinckley et al's later work examining the use of actions, such as lifting the phone to the ear to answer a call. These works point to the potential uses of tilt sensors in phones – an intuitive, button-free interaction model for many common tasks.

Of particular relevance to our work are two techniques for text entry that use tilt information. Both of these techniques focus on very small devices lacking a large number of buttons, and were not optimized or evaluated for speed of entry. *Unigesture* (Sazawal, Want et al. 2002) used tilt as an alternative to button pressing, eliminating the need for buttons for text entry. Rather than having the user make one of 8 ambiguous button presses (as is the present case with mobile phones), *Unigesture* has the user tilt the device in one of 7 directions to specify the group, or "zone", of the character that is desired. The ambiguity of the tilt is then resolved by using dictionary-based disambiguation.

TiltType (Partridge, Chatterjee et al. 2002) refines *Unigesture* by adding the combination of button pressing and tilt for entering unambiguous text. *TiltType* was designed to enter text into a small, watch-like device with 4 buttons. Pressing a button triggered an on-screen display of the characters that could be entered by tilting the device in one of eight directions, the appropriate tilt was then made, and the button released.

3 Chording Input for Mobile phones

Clearly, the issue of fast entry of text into mobile phones is still an open one. Though there have been attempts to replace the mobile-phone keypad with one that better lends itself to text entry, these efforts have gone unrewarded in the marketplace. As we have seen, much of the research in this area assumes an unchanged keypad for the phone. Because of this restriction, multiple keypresses are always required for unambiguous text: one press to select a letter grouping, and subsequent presses to select the letter within that grouping. Research has focused on attempting to reduce the number of disambiguating keystrokes required to enter a character of text, often by making assumptions about the corpus from which the user is selecting each word.

Though we accept the restriction of the traditional mobile-phone keypad, we have not joined the search for the minimal KSPC rate. Instead, we sought to apply the principle of chording to the question of mobile-phone text entry. As we have seen, chording reduces the number of keys needed to enter text by requiring the user to press multiple keys simultaneously – this seems to lend itself naturally to traditional mobile phones, where the number of keys is reduced by design.

Two possible chording implementations seem immediately obvious when considering mobile phones: 1: eliminate the keypad entirely, and replace it with a traditional one-handed chording keypad, and 2: require simultaneous presses of keys on the traditional keypad to enter a character. Given the limited success of alternative keypads, we immediately rejected any system that replaced the traditional layout. We also rejected the second option, since the limited size of the keypad makes simultaneous button pressing difficult, and any scheme that is an attempt to reduce this awkwardness would require that the character/key assignments be changed. We then considered a third option: maintaining the same key assignments, but adding keys that are used only for disambiguation of key presses from the traditional keypad. These disambiguating keys would be pressed simultaneous to keypad presses, and so would allow for concurrent between and within-group character selection.

In order to make this new technique simpler for experienced users, we sought to design it in such a way that the between-group selections would be made in the same way as other techniques. In order to maximize a transfer of skills, we wished to have the new, within-group selection keys used by the non-dominant hand, while the user pressed the mobile-phone keys with their dominant hand. This way, any technique they had learned for entering text with one hand could be directly applied to our chording technique, and speed learning.

We designed a new technique, dubbed *ChordTap*, where the mobile phone is augmented with three additional chording keys on the back side of the display (Figure 3-1). Users press a key with their dominant hand on the standard mobile phone keypad to select between groups of characters, while concurrently using their other hand to press the chording keys to make the within-group character selection.

Figure 3–1: ChordTap prototype. The right image shows the chord keys mounted on the back of the phone.

This technique is similar in theory to the consecutive press *two key* method discussed previously. *ChordTap* improves upon this by adding dedicated "chord" keys for making the within-group selection. With these extra keys, users can concurrently make between and within group selections, potentially improving entry speed.

There are three major design issues to consider in implementing *ChordTap*: where to place the chords, which chord combinations indicate which letter, and which key presses to consider as "events" for text entry.

3.1 Placing the Chords

The placement of the chord keys on the back of the display was done to facilitate use by the non-dominant hand, and shown in Figure 3-2. The user can place her thumb over the earpiece of the display, while comfortably resting the remaining fingers on the chord keys. In this configuration, the user then uses the dominant hand to press the regular keys on the mobile phone. Our design was for two hands in order to enhance speed, and simplify learning and use. Designing a key/chord layout to allow efficient one-handed text entry would likely be necessary for a commercial implementation of *ChordTap*.

Figure 3–2: ChordTap, as used by a left-handed (left) and right-handed user

3.2 Mapping Chords States to Within-Group Selection

Each key on a mobile phone has mapped onto it one, four, or five characters: some have only the numeral, most have three letters and one numeral, while the 7 and 9 keys have four letters and one numeral (Table 3-1). We used simple binary state switches for the chording keys. When designing *ChordTap*, we had to decide how many chording keys to have, and how to assign combinations of chords to particular character selections. The need to map five possible within-group character selections onto the chord states dictated that we would need at least 3 chording keys to ensure unambiguous selection. The chords' states can be viewed as 3-digit binary numbers, where the ith digit indicates whether that key is depressed ("1") or released ("0"). Table 3-1 illustrates.

Chord	***Character***	***Example***
000	Numeral	7
001	First letter	p
010	Second letter	q
100	Third letter	r
011	Fourth letter	s
101	Fourth letter	s
110	Fourth letter	s
111	Fourth letter	s

Table 3-1: Mapping of chord state to within-group characters. Example selection shown based on pressing the 7 key.

This mapping was chosen with the intent that it be as simple as possible for the user. We believe that pressing the first chord for the first letter, second chord for the second letter, and third chord for the third letter would be a fairly intuitive mapping. The choice to use all remaining chordings for the fourth letter was made because we felt that since this mapping was used least frequently, and it was not in keeping with the more frequently used i^{th} chord to i^{th} letter mapping, it would reduce errors & learning time to simply map them all to the fourth letter. One could alternatively envision using these remaining mappings for additional characters in a non-English alphabet.

3.3 EVENT HANDLING

To enter each character, the user must input precisely two pieces of information: the between-group selection using the standard keypad, and the within-group selection using the chords. Since both the within and between group selections are explicit but separate key presses, a number of options are available when determining exactly *when* a character should be generated. There are three apparent events that could be used to generate characters: chord presses, button presses, or both.

3.3.1 Treating Only Chord Presses as Events

In this implementation, chord presses trigger new text, but keypad presses do not. The keypad states are read only when an event is triggered by a chord press. As shown in Table 3-2, this approach saves work when two subsequent characters are present in the same letter group (i.e., on the same key). This savings is achieved because the user can hold down the same key while consecutively pressing the appropriate chords to generate the desired characters.

Key Held Before Action	*User Action*	*Key Held After Action*	*Output Text*
-	depress "6"	6	
6	depress and release 3rd chord	6	o
6	depress and release 2nd chord	6	n
6	release "6"	-	
	depress "5"	5	
5	depress and release 3rd chord	5	l
5	release "5"	-	
-	depress "9"	9	
9	depress and release 3rd chord	9	y

Table 3-2: Sequence of actions required to enter the string "only" in a ChordTap implementation that treats only chord presses as events. Some consecutive actions are combined because they either generate no text, or the same text is generated with either ordering.

Of the 36^2 possible pairs of consecutive characters, there are 112 ($6 \times {}^4P_2 + 2 \times {}^5P_2$) sequences that come from the same key. This means that for 9% of all pairings the user would not need to move their finger between character entries. Though these sequences

are not uniformly probable when entering text in a particular language, it is probable that there is savings in most languages.

3.3.2 Treating Only Keypad Presses as Events

In this implementation, keypad presses trigger new text to be entered into the phone, but chord presses do not. The chords' states are read only when an event is triggered by a keypad press. As demonstrated in Table 3-3, this approach to text entry gives a savings of work whenever two subsequent characters appear on different keys, but share the same chord.

Chord State Before Action	*User Action*	*Chord State After Action*	*Output Text*
000	depress 3rd chord	100	
100	depress and release "6"	100	o
100	release 3rd chord	000	
000	depress 2nd chord	010	
010	depress and release "6"	010	n
010	release 2nd chord	000	
000	depress 3rd chord	100	
100	depress and release "5"	100	l
100	depress and release "9"	100	y

Table 3-3: Sequence of user actions required to enter the string "only" in a ChordTap implementation that treats only keypad presses as events. Some consecutive actions are combined because they either generate no text, or the same text is generated with either ordering.

Of the 36^2 possible pairs of sequential characters, there are 262 (${}^{10}P_2 + 3 \times {}^{8}P_2 + 2 \times {}^{2}P_2$) sequences that share the same chording for both characters. This means that for 20% of all pairings the user would not need to change the chording between key presses, thus saving time.

3.3.3 Both Chord & Keypad Presses as Events

In this implementation, either a chord or keypad press results in new text being entered. The advantage of this implementation is that because every state change generates a new character, expert users would benefit from the savings illustrated in both the previous event handlers. In order for this implementation to work, we must assign no character

mapping to the "000" (all un-pressed) state of the chords. Table 3-4 demonstrates how fewer distinct actions are required to generate text in this configuration.

Chord State / Key Held Before Action	*User Action*	*Chord State / Key Held After Action*	*Output Text*
000 / -	depress "6"	000 / 6	
000 / 6	depress and release 3rd chord	000 / 6	o
000 / 6	depress and release 2nd chord	000 / 6	n
000 / 6	release "6"	000 / -	
000 / -	depress 3rd chord	100 / -	
100 / -	depress and release "5"	100 / -	l
100 / -	depress and release "9"	100 / -	y

Table 3-4: Sequence of actions required to enter the string "only" in a ChordTap implementation that treats both chord and keypad presses as events. Note that ordering of events required to enter text is not unique

This approach gives some savings for approximately 29% of all the possible sequences of two characters. However, this is likely more difficult to learn.

3.4 PROTOTYPE

We implemented a prototype to test the ChordTap technique. We selected the "keypad presses as events" approach for our prototype, since it had the greater savings of the single-event approaches, and because it is the traditional event for character generation in mobile phones.

3.4.1 Hardware

We used a Motorola i95cl phone. Chords were implemented by attaching momentary switches to the back of the phone, and connecting them via a mouse circuit board to the phone's serial port (see Figure 3-3). The apparatus used to detect movement was removed from the mouse, so all data packets sent were for changes in the chord states. Interpretation of the state-change data and key presses for text entry was done in software.

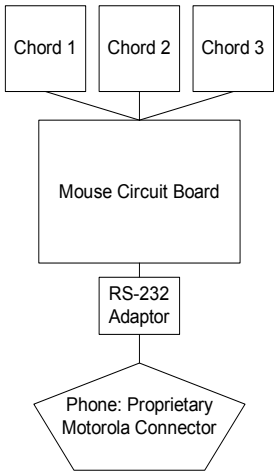

Figure 3–3: circuit diagram of ChordTap prototype.

3.4.2 Software

The software to read chords' states and render text was written in Java 2 Micro-Edition using classes from both the Mobile Devices Information Profile (MIDP 1.0) and proprietary i95cl specific classes. The *ChordTap* engine was written into an extension of the Motorola GUI "TextField" class, co that *ChordTap* enabled text fields could be inserted into any GUI built with the Motorola framework (see Figure 3-4).

Figure 3–4: ChordTap text field within Motorola GUI form.

4 User Study Comparing Early Learning Stage of ChordTap and MultiTap

Once the prototype was built, we endeavoured to test it by comparing user performance of *ChordTap* to *MultiTap*.

4.1 Goals

For this experiment, we chose *MultiTap* as the comparison technique, because it has served as a baseline in almost every other evaluation of text entry reported to date (Silfverberg, I. Scott MacKenzie et al. 2000; MacKenzie, Kober et al. 2001; Soukoreff and MacKenzie 2002), and because it is the most common of the consecutive action techniques. In previous experiments reported in the literature (Wigdor and Balakrishnan 2003), *MultiTap* users were usually instructed to use only the thumb on the dominant hand to press keys. However, informal observations of *MultiTap* users indicates that many use two thumbs to enter text. Since *ChordTap* is also a two-handed technique, we tested both one and two-handed *MultiTap* use. The one-handed case served as a common baseline for comparison with previous studies.

We wished to compare *ChordTap* with *MultiTap* in two areas: speed of entry, and frequency of error. Both of these were measured and reported herein.

4.2 Apparatus

All software, including those implementing the text entry techniques, data presentation, and collection software ran on the phone. No connection to an external computing device was used. Figure 4-1 shows the emulated screen of the phone as the user saw it during the experiment.

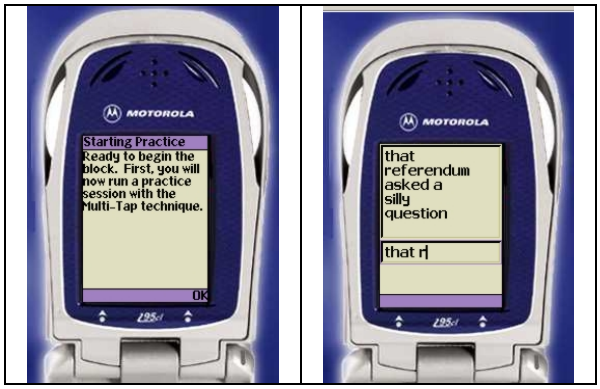

Figure 4–1: Emulation of phone as shown to user. Example instructions (left), and timed text entry portion (right).

Our *MultiTap* implementation used the i95cl's built-in *MultiTap* engine, with a 2 second timeout and timeout kill. We only considered lowercase text entry in this evaluation. As such, the *MultiTap* engine was modified slightly to remove characters from the key mapping that were not on the face of the key, so that the options available were only the lower case letters and numeral on the key. This matches the traditional *MultiTap* implementation in past experiments, such as *LetterWise* (MacKenzie, Kober et al. 2001).

4.3 PARTICIPANTS

Fifteen participants volunteered for the experiment. They were recruited from within the university community. Participants were generally students in undergraduate courses, or members of our lab. There were 5 women and 10 men of whom 2 were left-handed and 13 were right-handed. Participants were pre-screened so that no one with any substantial experience composing text using a mobile phone was included. Participants did not receive any tangible compensation for their participation.

4.4 PROCEDURE

Participants entered short phrases of text selected from MacKenzie and Soukoreff's corpus (MacKenzie and Soukoreff 2003). The corpus is a collection of 500 phrases varying in length from 18 to 32 characters, and is made up mostly of standard English words. The phrases include such things as "that referendum asked a silly question", "you

are not a jedi yet", and "my bank account is overdrawn". All punctuation was removed from the set, and all letters were changed to lower case.

These phrases were selected because they have been used in previous text entry studies involving *MultiTap* (MacKenzie, Kober et al. 2001; Wigdor and Balakrishnan 2003), allowing comparisons with this previous work. This corpus' high correlation of frequencies of letters to the English language is an asset, although it does not take into account abbreviations commonly used in mobile text input.

Timing began when participants entered the first character of the phrase, and ended when the phrase was entered completely and correctly. If an erroneous character was entered, the phone alerted the user by vibrating, and the user was required to correct their error. With this procedure, the end result is error-free in the sense that the correct phrase is captured. Also, the phrase completion time incorporates the time taken to correct for errors.

Phrases were shown to participants on the phone's display. Before beginning each treatment, participants were told to read and understand the displayed phrase before entering it, and were given instructions for that treatment as follows:

One-handed *MultiTap* instructions: to enter a character using the *MultiTap* technique, first find the key that is labelled with that character. Press that key repeatedly until the desired character is reached. Press once for the first character, twice for the second, three times for the third, and, if present, four times for the fourth. Once you have found the correct letter, and are ready for the next one, you simply repeat the process. If the letter you wish to enter next is on the same key, you must first either press the "right" arrow on the phone or wait two seconds for the cursor to advance. Please use only the thumb of the hand with which you hold the phone, and do not change hands during the experiment.

Two-handed *MultiTap* instructions (the same instructions were given as for the one-handed technique, with the following addendum): in this experiment, we are interested in seeing how people use MultiTap with two thumbs simultaneously. Please hold the phone

with two hands so that you are able to reach all of the keys with either thumb comfortably. As you enter text, use whichever thumb you wish to press the appropriate key – do whatever feels best for you. Feel free to change how you press keys as you get more comfortable with the technique, but please be sure to press only with your thumbs.

Figure 4–2: MultiTap as used with left, right, and both hands.

ChordTap instructions: to enter a character using the *ChordTap* technique, first find the key that is labelled with that character, then hold it down. Next, press the chord on the back of the display that corresponds to the position of the letter on the key. For the first letter, press the top chord, for the second letter, the 2nd chord from the top, for the 3rd letter, the 3rd chord from the top. To enter the 4th letter on a key, press any two of the chords. *ChordTap* works by detecting the state of the chords at the time you release a key. Because of this, you can continue to hold down a chord if two keys in a row require the same chord. It's also not important whether you press the chords before or after the key, just so long as the correct chord is being held when you release the keys.

The experimenter then demonstrated the relevant technique. To ensure that participants understood how the technique worked, they were asked to enter a single phrase that would require the use of all chord combination for *ChordTap*, or two successive letters on the same key for *MultiTap.*

Instructions were also given to describe space and delete keys, as well as to enter an extra space at the end of the phrase to indicate completion. The process for error correction

was also explained. Participants were directed to rest as required between phrases, but to continue as quickly as possible once they had started entering a phrase.

4.5 DESIGN

Data was collected for both one and two-handed *MultiTap* and *ChordTap*. To prevent the transfer effects between techniques inherent in within-subjects designs, a between-subjects design was used. Participants were randomly assigned to three groups of five. The first group performed the experiment with the one-handed *MultiTap* technique, the second group used the two-handed *MultiTap* technique, and the third group used the *ChordTap* technique.

Participants were asked to complete two sessions of 8 blocks of trials each. Each block required the entry of 2 identical practice phrases, followed by 20 different phrases selected randomly from the corpus. Phrase selection for each of the 16 blocks were done before the experiment, and presented in the same order to each participant. Phrases were selected such that all blocks had similar average phrase lengths. The same set of phrases and blocks were used for all three techniques. In other words, all participants entered identical phrases in the same order, the only difference being which technique they used. Participants were asked to rest for at least 5 minutes between each block, and each session of 8 blocks was conducted on separate days. In summary, the design was as follows:

3 techniques x
5 participants per technique x
2 sessions per participant x
8 blocks per session x
20 phrases per block (excluding practice phrases)

= 4800 phrases entered in total.

4.6 RESULTS

The data collected from 15 participants took an average of 9.9 minutes per block. A total of 109020 correct characters of input were entered for the 4800 phrases.

4.6.1 Physical Comfort

Some participants reported that their thumb became sore while using the one-handed *MultiTap* technique. When this was reported, the participants were encouraged to rest until they felt comfortable to proceed. No participant reported pain or discomfort in their wrist or arms.

4.6.2 Overall Entry Speed

The standard WPM (words-per-minute) measure was used to quantify text entry speed. Traditionally, this is calculated as characters per second * 60 / 5. Because timing in our experiment started only after entering the first character, that character should not be included in entry speed calculations. Thus, the phrase length is n-1 characters in these computations. Although users entered an extra space at the end of each phrase to signify completion, the entry of the last real character of the phrase denotes the end time.

The average text entry speeds for all blocks were 13.59 WPM for *ChordTap*, 10.11 WPM for one-handed *MultiTap*, and 10.33 WPM for two-handed *MultiTap* (Figure 4-3). Analysis of variance showed a significant main effect for technique ($F_{2,12} = 615.8$, $p < .0001$). Pair wise means comparisons showed that *ChordTap* significantly faster than both *MultiTap* techniques, but there was no significant difference between the one-handed and two-handed *MultiTap* techniques. Overall, *ChordTap* was 32% faster than two-handed *MultiTap*, and two-handed *MultiTap* was 2% faster than one-handed *MultiTap*.

Interestingly, we see in Figure 4-3 that while the progress of average speed per block for one-handed *MultiTap* fits the power law of learning with a high correlation (R^2 of .9032), this correlation for two-handed *MultiTap* is not as strong ($R^2 = .7964$). We attribute the

difference to users' changing their use of the non-dominant hand throughout the experiment in the two-handed case. Since hand use was not prescribed, users were free to change how it was used over the course of the experiment.

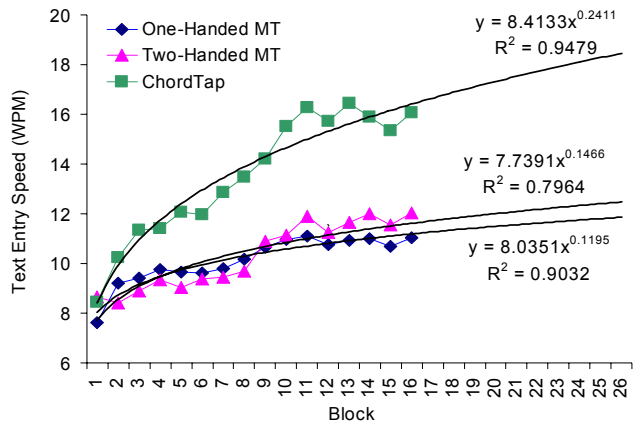

Figure 4–3: Entry speed (WPM) by technique and block for entire experiment. Best-fit power law of learning curve shows projected progress beyond the 16 blocks of measured data.

4.6.3 Learning

As Figure 4-3 illustrates, although all three techniques started out with roughly the same performance (average speeds of 7.62 WPM for one-handed *MultiTap*, 8.67 WPM for two-handed *MultiTap*, and 8.46 WPM for *ChordTap)*, *ChordTap* users improved the most over the course of the experiment, with an overall improvement of 90% between the first and last block averages, vs. 45% and 39% for one and two-handed *MultiTap* respectively. Two-handed *MultiTap* users spent most of the first day (first 8 blocks) with lower average speeds than the one-handed users. The one-day break seemed to benefit them more, as they spent all of the second day with higher speeds than the one-handed users.

	1-MultiTap	*2-MultiTap*	*ChordTap*
Day 1 Average	9.4	9.5	11.5
Day 2 Average	10.9	13.0	15.7

Table 4-1:Average speed (WPM) for each technique over the two days of the study

By the end of the experiment, one-handed *MultiTap* users had an average speed of 11.05 WPM, two-handed MultiTap an average speed of 12.04 WPM, and ChordTap an average

speed of 16.06 WPM. There was a significant main effect of technique on speed ($F_{2,12}$ = 123.67, p < .0001).

4.6.4 Error Rates

Recall that that our experimental procedure required participants to make corrections as they proceeded, with an end result of a completely correctly entered phrase. As such, the entry speed results discussed previously incorporate the cost of error correction. However, it is still helpful to look at a more explicit error rate. We calculate percentage error rate as the number of characters entered that did not match the expected character, divided by the length of the phrase. In this case, we used the actual length of the phrase, and not (n-1) as in the WPM rate.

Overall, there was a significant main effect for error rate ($F_{2,12}$ = 79.91, p < .0001). The error rate for one-handed *MultiTap* was 2.6%, two-handed *MultiTap* was 4.6%, and *ChordTap* was 3.3% (Figure 4-4).

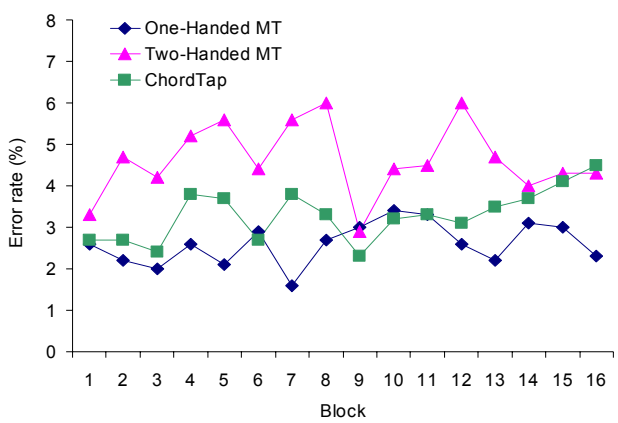

Figure 4–4: Error rate per 100 attempted character entries by block for all three techniques.

As seen in Figure 4-4, none of the techniques saw any measurable improvement in its error rate with practice. Analysis of variance showed only weak effect for block ($F_{15,12}$ = 1.81, p = .0283) and block x technique interaction ($F_{15,12}$ = 1.80, p = .0294).

With *ChordTap*, there are two ways that an incorrect character can be generated: incorrect key pressed (key error) or incorrect chording (chord error). As is illustrated in

Figure 4-5, an examination of error rates on individual letters shows that the key error rate is fairly consistent across letters (average rate of 1.8 key errors per 100 entries, standard deviation of 1.4). The chord error rate, however, varies more widely (average rate of 2.6 per 100 entries, standard deviation of 3.3).

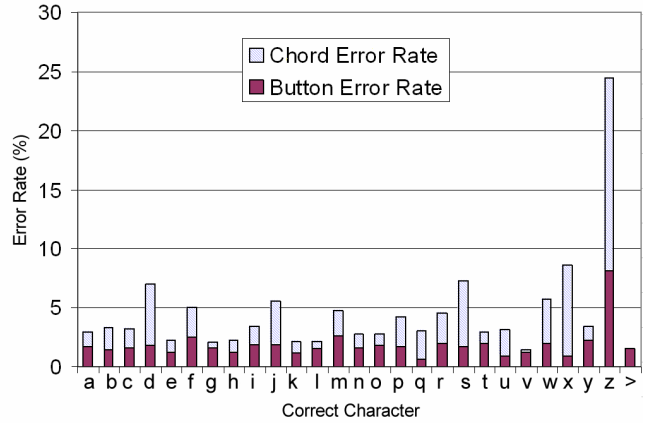

Figure 4–5: Key and Chord error rates per 100 attempted entries for each character in the experiment (space shown as “>”).

Pair wise means comparisons revealed that the chord error rate was significantly higher ($p < .0001$) for characters that required multiple-chord chording (s,z), as illustrated in Figure 4-5. The ‘s’ character was included in this pairing because of its significantly higher error rate than those characters with similar frequencies, and therefore practice time, in the experiment: {a,i,n,r}. We attribute this higher rate to the less obvious chording scheme (others are first chord=first letter, second chord = second letter, etc), and to the requirement to press two chords simultaneously.

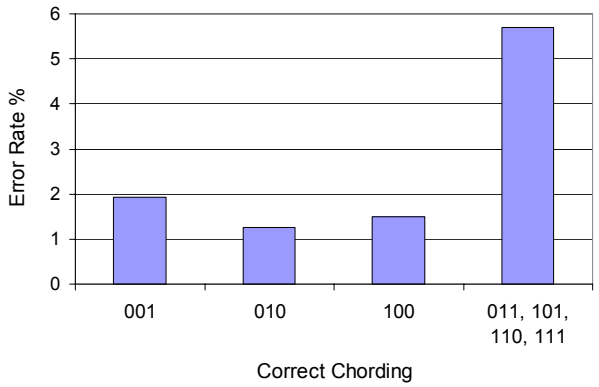

Figure 4–6: Chord error rate by required chord. Since all multiple-chords (011,101,110,111) produced the same letter in our prototype, they are combined in this graph.

4.7 DISCUSSION

This is a proof of concept experiment that indicates concurrent chording to be a viable text input technique for mobile phones. Note that these results were achieved despite a fairly crude prototype of switches for entering chords. As such, it is highly probable that with better industrial design of the chord switches and their integration with the phone, even greater performance benefits could be realized. It is also plausible that an appropriately designed layout could enable chording and keypad entry to be performed using the fingers of one hand, as the Septambic Keyer and Twiddler have shown (Figure 2-5).

5 A New Technique: TiltText

The success of *ChordTap* demonstrated the effectiveness of a chording approach to text entry into mobile phones. Its success is somewhat marred, though, by its reliance on two-hands for quick text entry. We wished to develop a fast chording technique that could be used with just one hand.

Though it might be possible to redesign the chording keys from *ChordTap*, we decided instead to investigate an alternative to these keys. We felt that there was value in eliminating the need for additional keys, and allow the user to concurrently make between and within-group character selections by doing some action other than pressing chord keys simultaneous to keypad keys.

Given the successes of Hinckley at al and Partridge et al in using tilt information in mobile devices, this seemed a natural information stream to replace the chording keypresses. In Partridge's *TiltType*, users pressed buttons and made tilting gestures consecutively to render text. We attempted to adapt this technique to allow for simultaneous tilt and press, effectively creating a chording keyboard with tilt taking the place of one of the keypresses.

Our technique, which we dubbed *TiltText*. The standard phone keypad mapping assigns three or four alphabetic characters, and one number, to each key. *TiltText* assigns an additional mapping by specifying a tilt direction for each of the characters on a key, removing any ambiguity from the button press. The user presses a key while simultaneously tilting the phone in one of four directions (left, forward, right, back) to input the desired character (Figure 5-1). For example, pressing the `2` key and tilting to the left inputs the character `a`, while tilting to the right inputs the character `c`. By requiring only a single keypress and slight tilt to input alphanumeric characters, the overall speed of text entry could possibly be increased. Further, by allowing using the tilt of the phone rather than the pressing of chord keys, a novice user is easily able to enter text using just one hand.

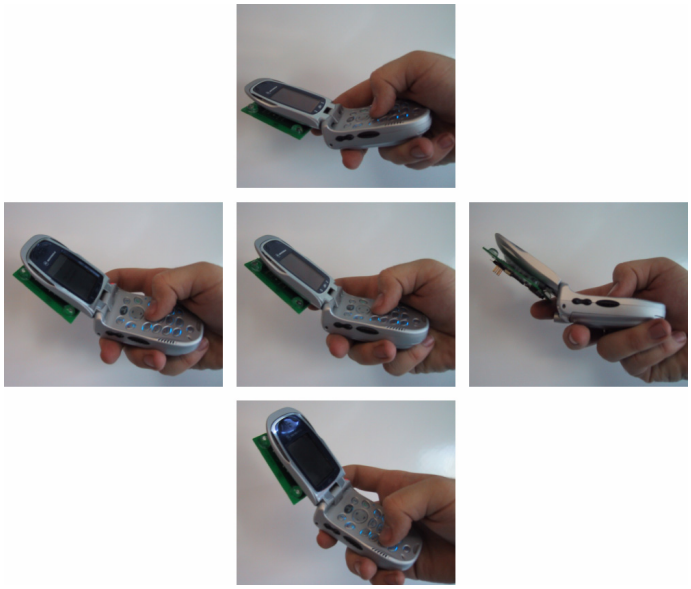

Figure 5–1: TiltText. The center picture shows the untilted phone where pressing a key enters its numeric value. Left picture: left tilt enters first character on key. Top picture: forward tilt enters second character. Right picture: right tilt enters third character. Bottom picture: tilting back (towards the user) enters fourth character if one exists for that key.

TiltType has the same root concept as our *TiltText* technique, in that tilt is used to disambiguate button presses.

Our present work builds upon *TiltType* in several significant ways. First, neither *TiltType* nor *Unigesture* were designed for use with mobile phone keypads, as we are proposing with our *TiltText* technique. We believe that using the standard mobile phone keypad will significantly increase the viability of tilting text input as a real, usable, technique. Second, while *TiltType* uses eight tilt directions, we only use a maximum of four tilt directions, reducing the accuracy demands on the user when tilting. Third, the algorithm used for detecting tilt in the *TiltType* technique is one which we dub *key tilt*, which, as is discussed later in our paper, is not the most optimal tilt detection mechanism for speedy text entry. We develop two alternative tilt detection mechanisms that improve upon *key tilt.* Finally, we present the results of a controlled experiment that provides the first set of usability data with regards to using tilt for text input.

5.1 Design Issues

TiltText uses the orientation of the phone along two axes to disambiguate the meaning of button presses. Tilting the phone to the left selects the first letter of the key, away from the body the second, to the right the third, and, if present, towards the body the fourth (see Figure 5-1). Pressing a key without tilting results in entering the numeric value of the key. Space and backspace operations are carried out by pressing unambiguous single-function buttons (as in *MultiTap*).

Supporting both lowercase and uppercase characters would require a further disambiguation step since a total of seven characters per key would need to be mapped for keys 2–6 and 8, and nine characters each for the 7 and 9 keys (Figure 5-2). Adding case sensitivity could be done by either requiring the pressing of a "sticky" shift-key, or considering the magnitude of the tilt as a disambiguator where greater magnitude tilts result in upper case letters, as Figure 5-2 illustrates. The latter technique, however, would likely make eyes-free entry more difficult.

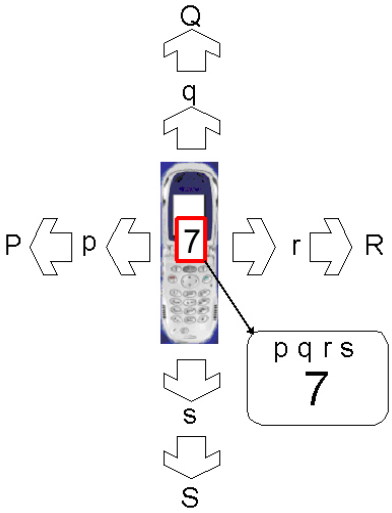

Figure 5–2: Uppercase text entry with *TiltText*. Tilting beyond a threshold makes the character uppercase.

5.2 TECHNIQUES FOR CALCULATING TILT

The tilt of the phone is taken as whichever direction has the greatest tilt relative to an initial "origin" value. After exploring various options during our development process, we have found that there are three main ways to determine the tilt value: *key tilt*, *absolute tilt*, and *relative tilt*.

5.2.1 Key Tilt

With this technique, first seen in the *TiltType* work (Partridge, Chatterjee et al. 2002), the amount of tilt is calculated as the difference in the value of the tilt sensors at key down and key up. This requires the user to carry out three distinct movements once the button has been located: push the button, tilt the phone, release the button. We conducted a pilot experiment comparing a *TiltText* implementation that used *key tilt*, and found that user performance with this implementation was much slower than the traditional *MultiTap* technique. For this reason, *key tilt* was not used in our final implementation.

5.2.2 Absolute Tilt

This technique compares the tilt sensor's value at any given time to a "fixed" absolute origin. Only two distinct movements are required to enter a character: tilt the phone, then press the key. In contrast, *key tilt* requires that the key be pressed first, phone tilted, then key released. However, absolute tilt is also not ideal, since in practice users do not maintain a constant arm posture. In order for the tilt value to be meaningful, the fixed origin will have to be reset every time the user's gross arm posture changes. Further, when using *TiltText* to enter two characters requiring tilt in opposite directions, more movement is required using this absolute approach, since the first tilt must be undone, then the new tilt applied. For example, entering the letters `ac` using the `2` key requires an initial tilt of some angle Δ to the left to enter the `a`. Then, the user has to tilt the same angle Δ in the reverse direction to return to the origin, before tilting another angle θ to the right to enter the letter `c`. The total amount of movement is $2\Delta + \theta$, instead of the smaller $\Delta + \theta$ that one may expect. However, one advantage of this method over *key tilt* is that if

successive characters with the same tilt direction are to be entered, then the user can keep the phone tilted at that direction for the successive key presses.

5.2.3 Relative Tilt

This approach calculates the tilt relative to a floating origin that is set when a tilt gesture begins. The beginning of a gesture is determined by continuously watching for a change in orientation or a change in the direction of a tilting gesture. This approach solves both problems of the absolute tilt method. Since all tilts are relative to the beginning of the gesture, there is no absolute origin that need be reset when changing arm position. Further, opposite direction tilts do not require double tilting, since the second tilt's origin is the end of the first tilt's gesture. So, entering the letters `ac` requires a tilt of some angle Δ to the left to enter `a`, then another tilt of angle θ to the right to enter the `c`, for a total movement of $\Delta + \theta$. Note that, just like with *absolute tilt*, when entering only letters, we can enter successive characters with the same tilt direction without re-tilting the phone, by looking at the last significant tilt.

5.3 PROTOTYPE

In order to evaluate and develop *TiltText*, we created a prototype implementation running on a commonly available mobile phone. Because of hardware limitations, we were forced to implement an absolute tilt system, rather than using the preferred relative tilt.

5.3.1 Hardware

We used a Motorola i95cl phone. The phone was equipped with an Analog Devices ADXL202EB-232 2-axis accelerometer board to enable tilt sensing, connected to the phone via a serial cable (with the addition of an external power line). Figure 5-3 is a circuit diagram of the TiltText prototype.

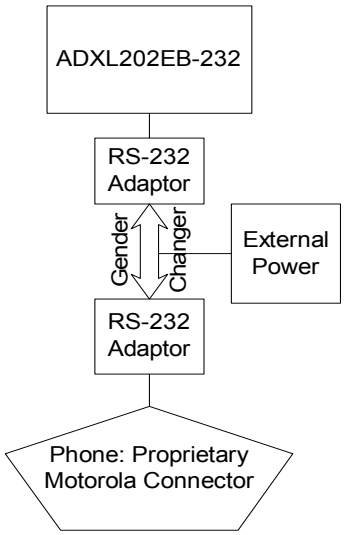

Figure 5–3: TiltText prototype circuit diagram.

An implementation of a relative tilt system would require regular sampling from the tilt sensor. Unfortunately, our hardware allowed only a reliable rate of ~10 Hz. Pilot studies, using a relative tilt implementation of *TiltText,* showed that participants' text entry speed increased to a point such that our mechanism for determining origins (repeated sampling of tilt at 10Hz) proved insufficient for determining tilt accurately. The resulting device-caused errors in recognition made entering text at higher speeds frustrating. From these pilot studies, we believe that rates of 20-50Hz would be required for a robust relative tilt implementation. As a result, despite the limitations discussed earlier, we had to implement an absolute tilt approach, allowing the user to reset the origin at any time by holding the phone at the desired orientation and pressing "0". The additional movement required by this approach, however, is acceptable for our evaluation purposes because if we can demonstrate that *TiltText* performs well despite this additional movement, then any more robust implementation using a relative tilt approach can only do better. In other words, our evaluation is biased against our new technique.

Because the ADXL board can detect a tilt of only fractions of a degree, only a very small tilt of the phone is necessary to disambiguate a button press. The maximum of the tilt in either axis was taken to be the intended tilt, with a 10% bias towards forward/back. This

bias is included based on our pilot studies which revealed a tendency to pitch to the dominant side when tilting forward with the wrist.

5.3.2 Handedness

Our prototype was implemented with the tilt sensor located at the centre of the axis of left/right rotation, and so we anticipated that there would be no difference in effectiveness for right and left-handed users. Figure 5-4 shows the prototype held in both the right & left hands.

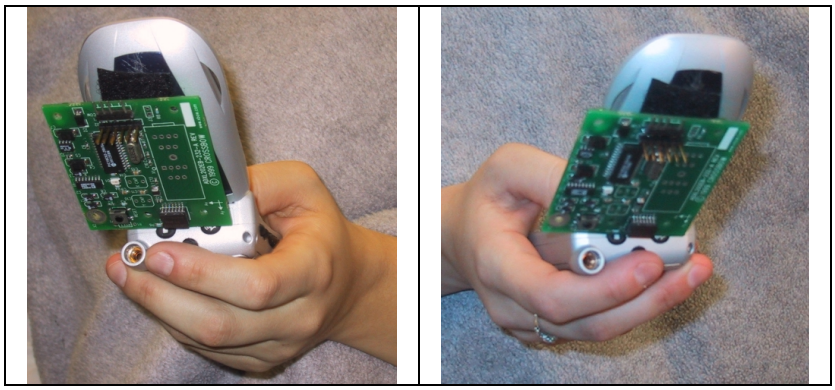

Figure 5–4: TiltText prototype held with left and right hands.

5.3.3 Software

The software to read tilts and render text was written in Java 2 Micro-Edition using classes from both the Mobile Devices Information Profile (MIDP 1.0) and proprietary i95cl specific classes. As was done for *ChordTap*, the *TiltText* engine was incorporated into a Motorola GUI TextField object, so that *TiltText* enabled text fields could be easily included in GUI's built using the Motorola framework (see Figure 5-5).

Figure 5–5: GUI built in Motorola framework, including a TiltText enabled field.

6 User Study Comparing Early Learning Stage of TiltText, MultiTap, and ChordTap

We wished to compare *TiltText* with *ChordTap*, and with other techniques previously reported. In order to do this, we conducted an experiment that was nearly identical to that used to evaluate *ChordTap*, so that the results could be compared directly.

6.1 Goals

We sought to compare the performance of *TiltText* to other techniques for text entry into mobile phones. For this first experiment, we chose as a comparison the most commonly implemented technique, *MultiTap*, because it is the common baseline in almost every other evaluation of text entry techniques reported to date (Silfverberg, I. Scott MacKenzie et al. 2000; MacKenzie, Kober et al. 2001; Soukoreff and MacKenzie 2002). As such, while the present study only directly compares *TiltText* to *MultiTap*, we can indirectly make comparisons of *TiltText*'s performance relative to other techniques via the results reported in the literature. We also intended to compare *TiltText* with *ChordTap* directly, and so the first-half (between-subject) of this experiment had a design identical to the *ChordTap* experiment. This allowed us to make a direct comparison between *ChordTap* and *TiltText*.

The experiment was conducted entirely on the mobile phone rather than simulating a mobile phone keypad on some other device. All software, including those implementing the text entry techniques, and data presentation and collection software used in the experiment ran on the phone. No connection to an external computing device beyond the tilt sensor was required. Of the major techniques for text entry into mobile phones evaluated in the literature over the last several years, this seems to be the first to do so on an actual mobile phone keypad / display. We believe that this helps to more closely represent real use, and so enhances the external validity of our experiment.

Our *MultiTap* implementation used the i95cl's built-in *MultiTap* engine, with a 2 second timeout and timeout kill. We only considered lowercase text entry in this evaluation. As such, the *MultiTap* engine was modified slightly to remove characters from the key

mapping that were not on the face of the button, so that the options available were only the lower case letters and numeral on the key. This matches the traditional *MultiTap* implementation in past experiments, such as *LetterWise* (MacKenzie, Kober et al. 2001) .

6.2 PARTICIPANTS

Ten participants volunteered for the experiment. They were recruited from within the university community. There were 5 men and 5 women of whom 3 were left-handed and 7 right-handed. Participants were pre-screened so that no one with any experience composing text using either technique was included. Participants did not receive any tangible compensation for their participation.

6.3 PROCEDURE

Participants entered short phrases of text selected from among those in MacKenzie's English phrase dictionary (www.yorku.ca/mack/phrases2.txt). These phrases were chosen because they have been used in previous text entry studies involving *MultiTap* (MacKenzie, Kober et al. 2001), allowing us to leverage this previous work. This corpus' high correlation of frequencies of letters to English is a benefit, although it does not take into account abbreviations commonly used in texting.

The desired text phrases were shown to participants on the screen on the phone. For consistency with past *MultiTap* experiments, participants were instructed to enter text only with the thumb of the hand with which they held the phone, and not to change hands during the experiment.

Timing began when participants entered the first character of the phrase, and ended when the phrase was entered completely and correctly. If an erroneous character was entered, the phone alerted the user by vibrating, and the user was required to correct their error. With this procedure, the end result is error-free in the sense that the correct phrase is captured. Also, the phrase completion time incorporates the time taken to correct for errors.

Before beginning each treatment, participants were told to read and understand the displayed phrase before entering it, and were given instructions for that treatment as follows:

MultiTap instructions: to enter a character using the *MultiTap* technique, first find the key that is labelled with that character. Press that key repeatedly until the desired character is reached. Press once for the first character, twice for the second, three times for the third, and, if present, four times for the fourth. Once you have found the correct letter, and are ready for the next one, you simply repeat the process. If the letter you wish to enter next is on the same key, you must first either press the "right" arrow on the phone or wait two seconds for the cursor to advance.

TiltText instructions: The technique works by tilting the phone in the direction of the letter you wish to enter, then pressing the key on which it is inscribed. For the first letter, tilt left. For the second letter, tilt forward. For the third letter, tilt to the right. For the fourth letter, tilt towards you. The direction of tilt is measured relative to the "centre" or "origin" position of the phone. You can reset the origin at any time by pressing the `0` key.

The experimenter then demonstrated the relevant technique. To ensure that participants understood how the technique worked, they were asked to enter a single phrase that would require tilting in all four directions for *TiltText*, or two successive letters on the same key for *MultiTap.*

Additional instructions were given for both techniques to describe space and delete keys, as well as to enter an extra space at the end of the phrase to indicate completion. The process for error correction was also explained to them. Participants were also directed to rest as they liked between phrases, but to continue as quickly as possible once they had started entering a phrase.

6.4 DESIGN

The ChordTap experiment reported previously used a between-subject design. Because we wished to examine the transfer effects between *MultiTap* and *TiltText*, we used a within-subject design for this experiment. In order to allow a direct comparison with the *ChordTap* results, the present experiment was designed such that each half of the experiment was identical to the *ChordTap* study. Thus, by examining only the *TiltText* data for those users who saw it first, we are able to make direct comparisons with *ChordTap*.

Participants were randomly assigned to two groups of 5 participants each. The first group performed the experiment with the *MultiTap* technique first, followed by *TiltText*, while the second group did it in the reverse order.

For each technique, participants were asked to complete two sessions of 8 blocks of trials each. Each block required the entry of 2 identical practice phrases, followed by 20 different phrases randomly selected from the corpus. The selection of phrases for each of the 16 blocks were done before the experiment, and presented in the same order to each participant. Phrases were selected such that all blocks had similar average phrase length. The same set of phrases and blocks were used for both techniques. In other words, all participants were required to enter identical phrases in the same order, the only difference being which technique they used first. Participants were asked to rest for at least 5 minutes between each block, and each session of 8 blocks each was conducted on separate days. At least 24 hours passed between sessions for the different techniques to limit interference. In summary, the design was as follows:

10 participants x
2 techniques (*MultiTap* and *TiltText*) x
2 sessions per technique x
8 blocks per session x
20 phrases per block (excluding practice phrases)
= 6400 phrases entered in total.

6.5 RESULTS

6.5.1 Data Summary

The data collected from 10 participants took an average of 10.3 minutes per block. A total of 145360 correct characters of input were entered for the 6400 phrases.

6.5.2 Physical Comfort

Some participants reported that their thumb became sore while using both techniques. When this was reported, the participants were encouraged to rest until they felt comfortable to proceed. No participant reported pain or discomfort in their wrist or arms.

6.5.3 Text Entry Speed vs MultiTap

We use the standard WPM (words-per-minute) measure to describe text entry speed. This is traditionally calculated as characters per second * 60 / 5. Because timing in our experiment started only after entering the first character, that character should not be included in calculations of entry speed. Thus, for the purposes of these computations, the length of a phrase is n-1 characters. Also, to signify completion, users had to enter an extra space at the end of each phrase. However, our timing considers the entry of the last real character of the phrase to be the end time.

The average text entry speed for all blocks were 11.76 WPM and 10.11 WPM for *TiltText* and *MultiTap* respectively. Overall, *TiltText* was 16.3% faster than *MultiTap.*

The means for the first block of trials were 7.42 WPM and 7.53 WPM, for *TiltText* and *MultiTap* respectively. Performance in both techniques increased steadily, with the means for the last (16th) block of trials of 13.57 WPM for *TiltText* and 11.04 WPM for *MultiTap*. While subjects performed marginally better with *MultiTap* initially, they improved considerably faster with *TiltText*, with the spread between the techniques reaching 22.9% in favour of *TiltText* by the end of the experiment. Analysis of variance indicated significant main effects for technique ($F_{1,8} = 615.8$, $p < .0001$), and block ($F_{15,120} = 145.2$, $p < .0001$) on speed. There was also a significant technique x block

interaction ($F_{15,120} = 20.5$, $p < .0001$) on speed, indicating that participants improved at different rates for the different techniques. Figure 6-1 illustrates these effects.

The use of a within-subjects design in our experiment raises the possibility of transfer effects. We attempted to balance these transfer effects by assigning subjects to two different groups, one group using *TiltText* first followed by *MultiTap*, and the other group doing it in the reverse order. However, as Poulton (Poulton 1989) points out, for this counterbalancing to work, any skill transfer that occurs must be symmetric. Analysis of variance showed a significant order x technique interaction ($F_{15,120} = 481.5$, $p < .0001$), indicating that our participants exhibited *asymmetric* transfer effects. This essentially means that participants' experience with the first technique they used affected their performance with the second technique by different amounts depending on whether the first technique was *TiltText* or *MultiTap*. As such, we did additional analyses by looking only at the data for the first technique participants performed (i.e., half the experimental data). This in effect results in a between-subjects design. Analysis of this data indicated effects similar to that of the full within-subjects data set: significant main effects for technique ($F_{1,8} = 120.9$, $p < .0001$), block ($F_{15,120} = 80.6$, $p < .0001$), and a significant technique x block interaction ($F_{15,120} = 14.5$, $p < .0001$). Figure 6-2 illustrates these effects.

From our analysis and Figure 6-2, we see that without prior experience with either technique, *TiltText* started out performing worse than *MultiTap,* only crossing over at block 4. This is likely because *TiltText* required participants to master two distinctly different motor skills: pressing the key, and tilting the phone. Whereas *MultiTap* required only a single type of motor action: multiple presses of the key.

Figure 6-3 shows data after participants switched techniques (i.e., the second half of the experiment). We see here that *TiltText* starts off faster than *MultiTap*, indicating that participants' were able to take advantage of and transfer their previous experience with *MultiTap* in the first half of the experiment. This is a positive indication since it means that real users with lots of experience with *MultiTap* can transfer at least some of that

skill if they switch to *TiltText.* Note, however, that there is quite a bit more variability in the performance for *TiltText*, as indicated by the poorer fit of the power curve as compared to Figures 4 and 5. This indicates that participants experienced some level of interference due to previous experience with *MultiTap*.

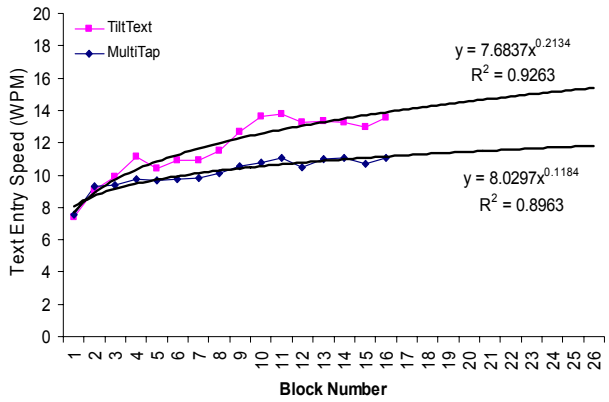

Figure 6–1: Entry speed (WPM) by technique and block for entire experiment. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks.

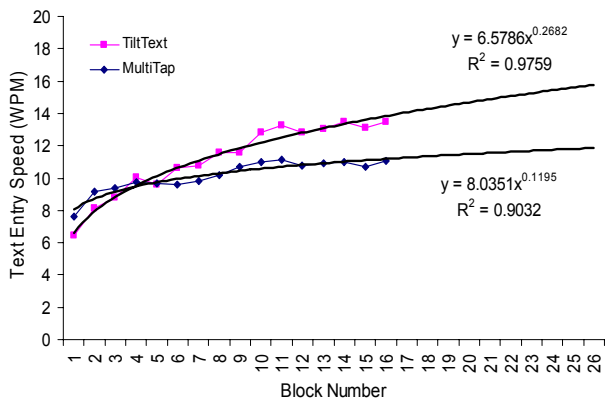

Figure 6–2: Entry speed (WPM) by technique and block, for the first half of the experiment, before participants switched techniques. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks.

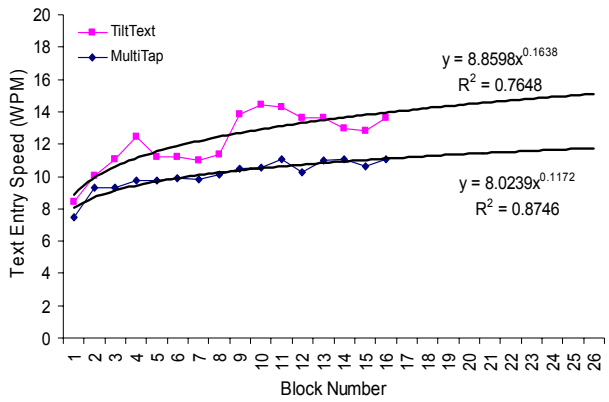

Figure 6–3: Entry speed (WPM) by technique and block, for the second half of the experiment, after participants switched techniques. Best-fit power law of learning curve shows projected progress beyond the measured data in the first 16 blocks.

As was expected, there was no main effect for handedness on speed of entry, though given the small number of left-handed users, our power to detect an effect was quite limited.

6.5.4 Text Entry Speed: TiltText vs ChordTap

The present experiment was identical to the one previously reported comparing *ChordTap* and *MultiTap*, with the addition of a second treatment for each participant. By disregarding data obtained during the application of the second treatment, and examining only the data for those who used *TiltText* first, we can conduct a direct comparison between *ChordTap* and *TiltText*. Figure 6-4 shows the data from the two experiments for ChordTap and TiltText entry speeds over the 16 blocks of the experiment:

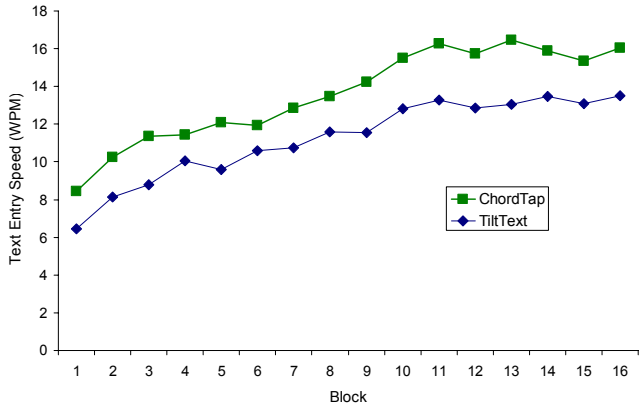

Figure 6–4: WPM rate of ChordTap vs TiltText over the 16 blocks of the experiment.

As the Figure 6-4 demonstrates, *ChordTap* users started off at a higher speed than did users of *TiltText*, and continued to have higher speeds over the course of the experiment. Interestingly, the difference between *ChordTap* and *TiltText* did not seem to grow significantly over the course of the experiment: on the first block, ChordTap was ~ 2 WPM faster, on the 16th, it was ~ 2.56 WPM faster. Figure 6-5 shows the difference in speed over the 16 blocks of the experiment:

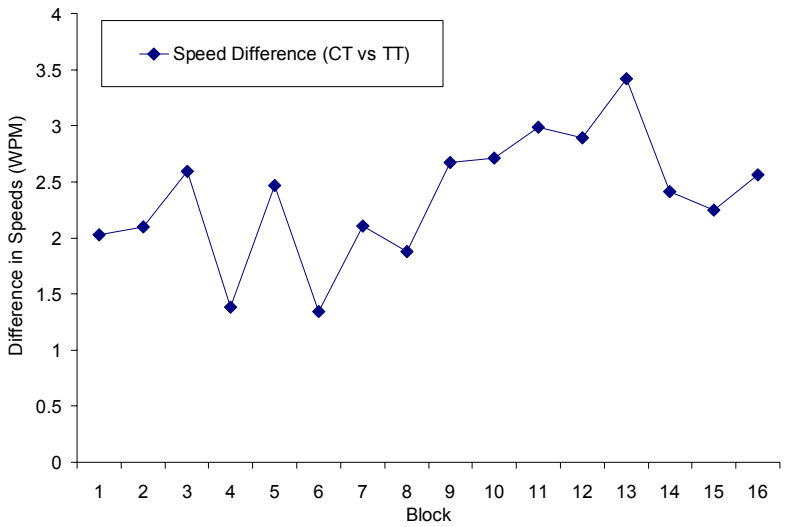

Figure 6–5: Difference in speeds between ChordTap and TiltText over the 16 blocks of the experiment.

Though the difference is significant ($F_{1,15} = 9.63$, $p < .0001$), its small rate of increase suggests that a relative-tilt implementation of *TiltText* could be as fast as or faster than our current *ChordTap* in future generations.

6.5.5 Error Rates: TiltText vs MultiTap

Given that our experimental procedure required participants to make corrections as they proceeded, with an end result of a completely correctly entered phrase, the entry speed results discussed previously incorporate the cost of error correction. However, it is still useful to look at a more explicit error rate. We calculate percentage error rate as the number of characters entered that did not match the expected character, divided by the

length of the phrase. In this case, we used the actual length of the phrase, and not (n-1) as in the WPM rate.

Overall, error rates were much higher for *TiltText* (11%) than for *MultiTap* (3%). This effect was statistically significant ($F_{1,8} = 1378.8$, $p < .0001$). There was also a significant effect for blocks ($F_{15,120} = 21.1$, $p < .0001$). A significant technique x block interaction ($F_{15,120} = 23.3$, $p < .0001$) and Figure 6-6 indicate that while the error rates for *MultiTap* remain quite constant throughout the experiment, the error rates for *TiltText* drop rapidly over the first 8 blocks, and begin to asymptote from block 9 onwards.

As with the entry time analysis, a significant order x technique interaction ($F_{15,120} = 168.9$, $p < .0001$) indicates that our participants exhibited *asymmetric* transfer effects. An analysis of the first half of the data (i.e., before participants switched techniques) indicates main effects similar to that of the entire dataset: technique ($F_{1,8} = 632.4$, $p < .0001$), blocks ($F_{15,120} = 7.3$, $p < .0001$), and technique x block interaction ($F_{15,120} = 10.4$, $p < .0001$), as figure 6-7 illustrates. Interestingly, the mean *TiltText* error rate (8.6%) was lower than for the entire data set, indicating that the lack of interference from *MultiTap* was beneficial.

Figure 6-8 illustrates the data from trials in the second half of the experiment (i.e., after participants switched techniques). Comparing this to Figure 6-7, we see that the mean *TiltText* error rate of 13.5% is much higher than the mean 8.6% rate in the first half of the experiment. Further, the first 8 blocks of trials did not exhibit a constant trend for *TiltText.* Clearly, participants' previous experience with the *MultiTap* technique was having a detrimental effect on their ability to use *TiltText* right after the switch in technique occurs. This is consistent with the effect observed in the text entry speed data illustrated earlier in Figure 6-3. However, this effect wears off roughly after block 8. This is interesting, since this is the last block completed before participants broke for the day, resuming 24 hours later with block 9. The interference effect drops after this break.

To examine the cause of the higher *TiltText* error rate, it is useful to group errors into two categories: *tilt errors* and *button errors*. *Tilt errors* are those where the participant entered a letter that appears on the same button as the correct letter, indicating that an erroneous tilt was made. *Button errors* are those where the participant entered a letter that appeared on a different button.

We had anticipated that *button errors* would have similar rates for *TiltText* and *MultiTap*. However, the results showed a significant difference ($F_{1,8}$= 320.67, $p < .0001$), where 3% of characters entered in the *MultiTap* trials were *button errors*, but only 1.5% of characters entered in *TiltText* showed this type of error.

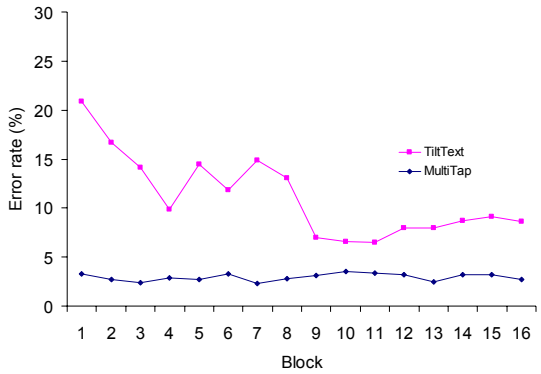

Figure 6–6: Error rates (%) by technique and block for entire experiment

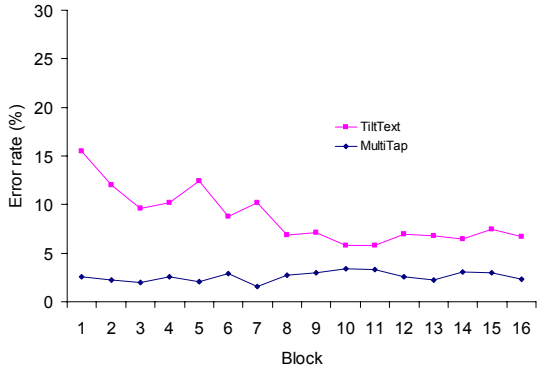

Figure 6–7: Error rates (%) by technique and block for the first half of the experiment, before participants switched techniques

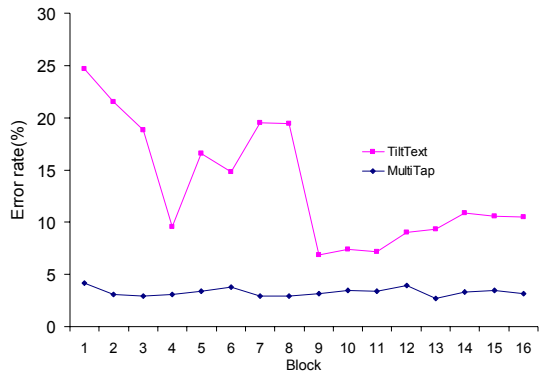

Figure 6–8: Error rates (%) by technique and block for the second half of the experiment, after participants switched techniques

Reconciling this low *button error* rate with the high overall error rate for *TiltText*, it is clear that most of the errors committed while using *TiltText* were *tilt errors*. Breaking down the *tilt error* rate by letter shows that participants committed significantly more errors for some letters than others ($F_{25,200,}$= 2.47, $p < .001$), as Figure 6-9 illustrates.

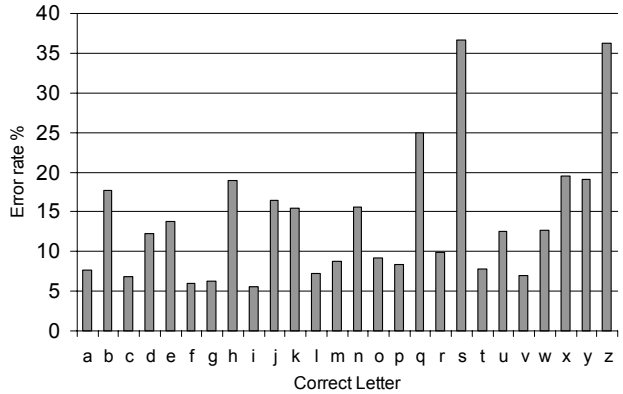

Figure 6–9: Tilt error rates (%) for each letter for the entire experiment.

Analysis of variance showed a significant main effect for tilt direction on *tilt error* rate ($F_{3,24}$= 37.6, $p < .0001$). Pairwise means comparisons showed a significantly higher *tilt error* rate for those letters requiring forward or backward tilting than those requiring right or left tilting. In particular, backwards tilt results in significantly higher errors than all the other tilt directions. Figure 6-10 illustrates this trend.

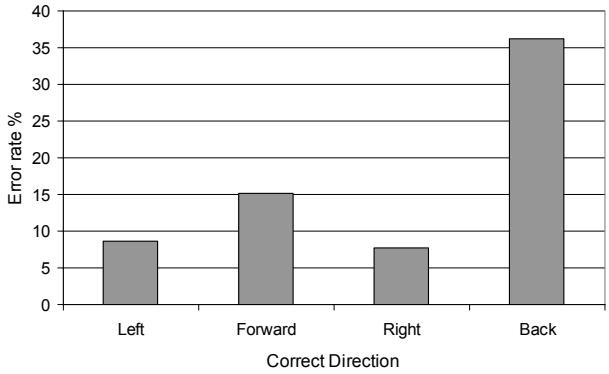

Figure 6–10: Error rates (%) by direction of tilt for correct letter for the entire experiment.

As was discussed previously, the overall *TiltText* error rate decreases with practice. As such, it is possible that the high *tilt error* rate for backward tilt (letters "s" and "z") is due to the limited amount of practice users had with entering the letter "z", which was only entered 3 times in the experiment by each participant. However, the other letter that required backward tilting, "s", also showed a similarly high *tilt error* rate, despite being entered very frequently during the experiment. In other words, additional practice did not seem to decrease the backward *tilt error* rate significantly, indicating that users had an inherent difficulty with backward tilting actions.

When participants committed a *tilt error* for a letter requiring a left or right gesture, 82% of the time they ended up entering the forward-tilt letter on that button. This indicates that the 10% bias we introduced in our algorithm seems to overcompensate. Reducing this compensation factor may lower *tilt error* rate for left/right tilts.

As was expected, there was no main effect for handedness on speed of entry, though given the small number of users, our power to detect an effect was quite limited.

6.5.6 Error Rates: TiltText vs ChordTap

TiltText showed a significantly higher error rate than *ChordTap*. Figure 6-11 demonstrates the differences over the course of the experiment (once again we are examining only the between-subject data for *TiltText*):

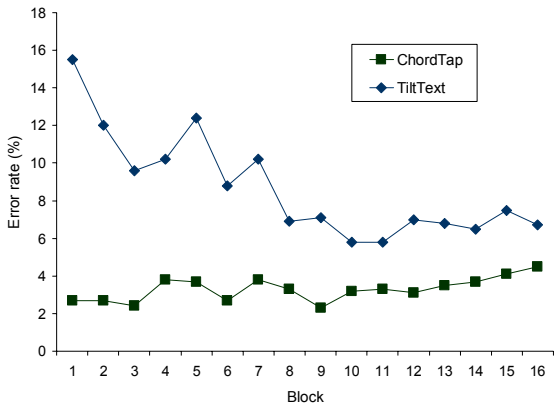

Figure 6–11: Error rate of TiltText vs ChordTap

The difference in error rates was significant ($F_{1,8} = 46.2$, $p < .0001$), though the gap was narrowing in the later stages of the experiment.

6.6 DISCUSSION

As discussed in (MacKenzie, Kober et al. 2001), learning of text entry techniques can be roughly divided into three stages: an initial discovery stage where users' performance is significantly affected by prior experience, a second stage where users are developing their motor skills with the new techniques, and a third stage where expert users perform all actions reflexively.

In our experiment, it is quite clear that learning is still continuing, even at the end of the experiment (figures 6-3 – 6-5), and thus the third stage is not reached. Most interestingly, we found that when participants did the experiment with *MultiTap* first, followed by *TiltText,* their experience with *MultiTap* very significantly affected their ability to use *TiltText* as evidenced by the higher errors rates. However, our analysis indicated that this prior experience did *not* affect the participants' entry speed when they switched to *TiltText.* Indeed, even though they made more errors, they were able to compensate for them by higher speeds of action. It appears from our analysis that participants move to the second stage at around block 9, or about 180 phrases (~4000 keystrokes).

One of the reasons why we chose to compare *TiltText* to *MultiTap* was because *MultiTap* is used as a baseline technique in most studies of text entry techniques. Comparing our results to that of (MacKenzie, Kober et al. 2001), we see approximately that the number of phrases entered per technique by block 16 in our experiment is roughly equivalent to the 6th or 7th session in MacKenzie et al.'s experiment. At this point, their data for *MultiTap* is roughly in the 11 WPM range, which is very close to ours. At the same point in time, entry rates for *LetterWise* are about 14 WPM, which is in the same range as our experiment's rate for *TiltText* of 13.57 WPM. While the different experimental designs, number of phrases per block, and other factors necessarily imply that these cross-experiment comparisons are not precise, this rough analysis does give us a ballpark sense of how *TiltText* compares to techniques other than *MultiTap.*

The increased error rate for forward/backward movements is possibly explained by limitations of the absolute tilt system used in our experiment. Participants tended to set the origin, then have the phone slowly "creep" forward as they made far more forward than back tilting gestures. As a result, the phone was always tilted somewhat forward. This meant that an exaggerated back gesture was required to enter "s" or "z", which users often failed to accomplish on the first attempt. The tendency to hold the phone in a forward position also explains why most *tilt errors* resulted in entering the forward tilt letter on the same button. Due to hardware constraints, we used an absolute tilt implementation of *TiltText* in our experiment, instead of the more efficient and accurate relative tilt method. We posit that error rates could be improved if we use relative tilt, since users would not have to tilt a specific amount past the origin between characters as required in the absolute tilt method, and they also would not have to exaggerate the back tilts to overcome the forward posture. These extra requirements are a likely cause of errors, particularly if users attempt to perform the technique quickly without watching the screen. Relative tilt would be more amenable to fast, "eyes-free", use.

Previous work (Card, Moran et al. 1980; MacKenzie, Kober et al. 2001; MacKenzie 2002; Soukoreff and MacKenzie 2002) present very detailed analyses with regards to the various components of text entry time using various existing techniques. We refer readers

to those papers for the details, however, it is worthwhile discussing how *TiltText* differs. First, all other techniques have only a single motor component: that of selecting the correct key and pressing it a certain number of times. In contrast, *TiltText* has two motor components: selecting and pressing the correct key as with other techniques, *and* tilting the phone in the appropriate direction. One could probably argue that there is also an additional cognitive component required in determining which direction to tilt, at least during the initial discovery and skill development stages of learning. This raises an interesting question with regards to the applicability of the standard metric for evaluating text entry techniques put forth by (MacKenzie 2002): Keystrokes Per Character (KSPC). *TiltText* by definition has a KSPC of 1, not including error correction. However, KSPC does not fully capture the second, tilting, aspect of *TiltText*. While the tilting movements required by *TiltText* are small, particularly if a relative tilt implementation is used, Fitts' law tells us that there is nevertheless a cost associated with this movement. It is difficult to determine exactly the parameters of the tilting movement in order to calculate a precise estimate of its cost, however, we can say that the tilt angle required ranges from a little more than 0 degrees to an approximate maximum of 90 degrees. From our observations of participants in our experiment, it appears that the average tilt angle is probably around 30 degrees. With a more definitive determination of this parameter or at least a smaller bound on its range, it would be possible to develop a model that more accurately describes *TiltText* than KSPC. Since there is no fixed target that the tilt must accomplish, a simple application of Fitts' law is not appropriate. However, the work of Accot and Zhai on the "steering law" (Accot and Zhai 1997) and "crossing based interfaces" (Accot and Zhai 2002) provide a foundation on which an analysis could be performed. We leave the development of such a model to future work, once a more detailed determination of the tilting parameter is performed.

7 Conclusions and Future Work

7.1 Conclusions

We have presented experimental work that investigates the effectiveness of the classic concurrent-press technique, chording, when applied to mobile phone text entry. The performance advantages demonstrated over the consecutive action *MultiTap* technique, and consequently the inherent advantages over linguistic disambiguation techniques, indicates that concurrent text input could be a viable alternative to current techniques. Specifically, despite a relatively crude prototype implementation, we have shown that our *ChordTap* system is significantly faster for input than *MultiTap*, with a 45% difference in by the end of our experiment. We have also shown that *ChordTap* is easier to use than *MultiTap*, as users started out slower, but improved much more quickly with *ChordTap* than with *MultiTap*, and users are less prone to make errors when using our *ChordTap* technique.

We have also demonstrated an alternative chording implementation: *TiltText,* which replaces chord keys with tilting gestures. Though *TiltText* was not as fast as *ChordTap* (16% slower), and was more prone to errors (9% as compared to 2.6%), it did show speed improvements over *MultiTap* (22% faster). Because *TiltText* requires the use of just one hand, and because of the other potential uses of tilt sensors in mobile devices (Harrison, Fishkin et al. 1998; Hinckley, Pierce et al. 2000; Hinckley and Horvitz 2001), it seems a more logical system to be adopted by a manufacturer.

7.2 Contributions

The primary contribution of this work is the introduction of concurrent pressing to mobile phone text entry. All prior research we are aware of focused on trying to reduce the number of keystrokes required to enter a character of text (MacKenzie, Kober et al. 2001; MacKenzie 2002; MacKenzie and Soukoreff 2002). By applying the chording principle of simultaneous pressing, we have demonstrated that faster text entry is possible without reducing the number of keystrokes required to enter text.

Within this new domain, we have contributed two new text entry techniques: *ChordTap*, and *TiltText*. Though both were implemented with fairly crude prototypes, we have shown them to be efficient and easily learned.

Lastly, this work was the first we are aware of that conducted text-entry analysis on actual mobile phones. Prior to this, new techniques were tested with either a computer keyboard or mobile phone keypad, with text displayed on a computer monitor (MacKenzie, Kober et al. 2001; MacKenzie and Soukoreff 2002). We feel that by conducting our experiments on mobile phones, we have shown how to enhance the validity of mobile-phone text entry experiments.

7.3 FUTURE DIRECTIONS

There are many aspects of concurrent-text entry to mobile phones that could yet be explored.

First, a theoretical analysis of *ChordTap* and *TiltText* performance, as was done for previous techniques (Silfverberg, I. Scott MacKenzie et al. 2000), would give us an idea of how best to improve each of the techniques.

Second, a re-examination of our experimental data, specifically examining the letter entered prior to the error, might help to reveal what proportion of errors for each of the techniques can be attributed to keypad pressing and chording technique specifically.

Third, a longer-term study of each of the techniques, implemented on phones that participants could use in their natural environment, would help to enhance the validity of the experiments.

There are also future directions of research that specifically apply to either *ChordTap* or *MultiTap*, which we will now examine individually.

7.3.1 ChordTap

This is a proof of concept experiment that indicates concurrent chording to be a viable text input technique for mobile phones. Note that these results were achieved despite a fairly crude prototype of switches for entering chords. As such, it is highly likely that with better industrial design of the chord switches and their integration with the phone, even greater performance benefits could be realized. It is also plausible that an appropriately designed layout could enable chording and keypad entry to be performed using the fingers of one hand. It will be the topic of future work to examine how this technique could be adapted for the use with one hand, and how varying the placement of the chords impacts speed of entry. We will also examine the application of the lessons of Gopher & Koenig (Gopher and Koenig 1983) to alter the mappings of chords to characters to optimize entry speed, such as mapping the index-finger chord to the more frequently used English characters.

7.3.2 TiltText

A new Motorola phone (i730), promises a higher refresh rate from the serial port, allowing a relative tilt implementation. We intend to attempt this, and hope that it will show even better results for speed of entry. We believe a relative tilt system will also reduce errors, but other work on reducing the error rate via possibly better sensing hardware and algorithms is clearly needed.

We will also examine whether different tilting directions are more natural for the wrist, and perhaps replace the simplistic “left/right, back/forward” technique examined here. We will also test a *TiltText* implementation that groups letters to minimize the need to re-tilt between character entries.

It is the goal of our future work with TiltText to make sufficient improvements in its implementation in the hope that performance will approach that of ChordTap.

8 Bibliography

Accot, J. and S. Zhai (1997). Beyond Fitts' Law: models for trajectory-based HCI tasks. ACM CHI Conference on Human Factors in Computing Systems, New York, NY, ACM.

Accot, J. and S. Zhai (2002). More than dotting the i's - foundations for crossing-based interfaces. ACM CHI Conference on Human Factors in Computing Systems, New York, NY, ACM.

Barfield, W. and T. Caudell, Eds. (2001). Fundamentals of Wearable Computers and Augmented Reality. Mahwah, New Jersey, Lawrence Erlbaum Associates.

Bartlett, J. (2000). Rock 'n' Scroll is here to stay. IEEE Computer Graphics and Applications May/June 2000: 40-45.

Card, S., T. Moran, et al. (1980). The keystroke-level model for user performance time with interactive systems. Communications of the ACM 23(7): 396 - 410.

Conrad, R. and D. Longman (1965). Standard typewriter versus chord keyboard: An experimental comparison. Ergonomics 8: 77-88.

Engelbart, D. (1986). History of personal workstations. Proceedings of the ACM Conference on The history of personal workstations: 73 - 83.

Engelbart, D. and W. K. English (1968). A research center for augmenting human intellect. Proceedings of the Fall Joint Computer Conference: 395-410.

Fishkin, K., A. Gujar, et al. (2000). Embodied user interfaces for really direct manipulation. Communications of the ACM 43(9): 75-80.

Gopher, D. and W. Koenig (1983). Hands coordination in data entry with a two-hand chord typewriter, Cognitive Psychology Laboratory, Dept. of Psychology, University of Illinois, Champaign, ILL 61820.

Gopher, D. and D. Raij (1988). Typing with a two-handed chord keyboard: Will QWERTY become obsolete. IEEE Transactions on Systems, Man, and Cybernetics 18(4): 601-609.

Grinter, R. E. and M. A. Eldridge (2001). Y do tngrs luv 2 txt msg? Proceedings of the European Conference on Computer Supported Cooperative Work - ECSCW 2001.

Harrison, B., K. Fishkin, et al. (1998). Squeeze me, hold me, tilt me! An exploration of manipulative user interfaces. ACM CHI Conference on Human Factors in Computing Systems, New York, NY, ACM.

Hinckley, K. and E. Horvitz (2001). Toward more sensitive mobile phones. ACM UIST Symposium on User Interface Sofware and Technology, New York, NY, ACM.

Hinckley, K., J. Pierce, et al. (2000). Sensing techniques for mobile interaction. ACM UIST Symposium on User Interface Software and Technology, New York, NY, ACM.

MacKenzie, I. S. (2002). KSPC (keystrokes per character) as a characteristic of text entry techniques. Fourth International Symposium on Human-Computer Interaction with Mobile Devices, Heidelberg, Germany, Springer-Verlag.

MacKenzie, I. S. (2002). Mobile text entry using three keys. Proceedings of the Second Nordic Conference on Human-Computer Interaction - NordiCHI, 2002: 27-34.

MacKenzie, I. S., H. Kober, et al. (2001). LetterWise: Prefix-based disambiguation for mobile text input. ACM UIST Symposium on User Interface Software and Technology, New York, NY, ACM.

MacKenzie, S. and W. Soukoreff (2002). A model of two-thumb text entry. Proceedings of Graphics Interface 2002: 117-124.

MacKenzie, S. and W. Soukoreff (2002). Text entry for mobile computing: Models and methods, theory and practice. Human-Computer Interaction 17: 147-198.

MacKenzie, S. and W. Soukoreff (2003). Phrase sets for evaluating text entry techniques. Extended Abstracts of the ACM CHI Conference on Human Factors in Computing Systems: 754-755.

Matias, E., I. S. MacKenzie, et al. (1993). Half-QWERTY: A One-handed Keyboard Facilitating Skill Transfer From QWERTY. roceedings of the ACM Conference on Human Factors in Computing Systems - INTERCHI '93: 88-94.

Matias, E., I. S. MacKenzie, et al. (1996). One-Handed Touch-Typing on a QWERTY Keyboard. Human-Computer Interaction 11: 1-27.

Norman, D. and D. Fisher (1982). Why alphabetic keyboards are not easy to use: Keyboard layout doesn't much matter. Human Factors 24: 509-515.

Partridge, K., S. Chatterjee, et al. (2002). TiltType: accelerometer-supported text entry for very small devices. ACM UIST Symposium on User Interface Software and Technology, New York, NY, ACM.

Poulton, E. C. (1989). Bias in quantifying judgements. Hillsdale, NJ, Lawrence Erlbaum.

Rekimoto, J. (1996). Tilting operations for small screen interfaces. ACM UIST Symposium on User Interface Software and Technology, New York, ACM.

Rosenberg, B. (1994). Chord Kyboards. London, UK, Queen Mary & Westfield College.

Sazawal, V., R. Want, et al. (2002). The Unigesture Approach. One-Handed Text Entry for Small Devices. Mobile HCI, Pisa, Italy.

Schmidt, A., K. Aidoo, et al. (1999). Advanced interactions in context. HUC'99: 89-101.

Silfverberg, M., I. Scott MacKenzie, et al. (2000). Predicting text entry speeds on mobile phones. ACM CHI Conference on Human Factors in Computing Systems, New York, NY, ACM.

Soukoreff, W. and I. S. MacKenzie (2002). Text entry for mobile computing: Models and methods, theory and practice. Human-Computer Interaction **17**: 147-198.

Ward, D. J., A. F. Blackwell, et al. (2000). Dasher—a data entry interface using continuous gestures and language models. Proceedings of the 13th annual ACM symposium on User interface software and technology: 129 - 137.

West, L. J. (1998). The standard and dvorak keyboards revisited: Direct measures of speed. Technical report, Santa Fe Institute.

Wigdor, D. and R. Balakrishnan (2003). TiltText: Using tilt for text input to mobile phones. Proceedings of the 16h Annual ACM Symposium on User Interface Software and Technology, ACM.

Wigdor, D. and R. Balakrishnan (2004 - in press). Comparison of Consecutive and Concurrent Input Text Entry Techniques for Mobile Phones. CHI Conference on Human Factors in Computing Systems: 8.

Zhai, S. and B. A. Smith (2001). Alphabetically biased virtual keyboards are easier to use - layout does matter. Extended Abstracts of CHI2001: 321□322.